Five Roles for Artforms

A Role-Stratified Type Theory with Traceable Gates

Sergey Krivov

October 2026

Abstract

The textbook *Artform Calculus* sorts the data of every construction into five roles, FORM, PRESENTATION, WITNESS, SCRIPT and RENDERING, and states a policy for them: what has been built, presented, certified or done may be used downstream; a witness or a script reaches the frame only through a gate that records what was assumed; and nothing may depend on a reading. This paper makes that discipline precise and tests the claim that five is the right number.

The policy is generated by four levels of a construction: the carrier, its carried structure, the instance, and the display. Among the policies that levels generate, it is the only one that meets four requirements the book states, and it is path closed, which is exactly the condition under which checking each direct dependency suffices. For constructions whose dependencies are graded *free*, *gated* or *absent* we prove the firewall, mediation by recorded gates, frame stability and presentation independence, for chains of dependencies and for computed values. The policy tells every two roles apart except WITNESS and SCRIPT, which share the instance level and must: a certificate may mention a move, and a guarded move takes its certificate. Their rules tell them apart. A history is a list of applications of scripts, and a certificate only admits an application; counting certificates as values, or applications by their endpoints, changes the counts the book makes, so a discipline that counts each role's data one way needs both roles. A state is a legal choice of values, which is witness data, so the five roles classify every datum. These results and the book's examples are checked in Lean 4 with Mathlib.

The second part gives $\mathsf{RATT}_{\mathrm{c}}$, a universe-free dependent calculus whose judgements carry the role. Every rule computes an effect in the min–max algebra of the three grades, and a registered gate consumes a stamped credential and contracts to a private node that records its event. We prove principal effects, substitution, subject reduction, strong normalization, confluence, canonical forms, and decidable conversion and type checking; static–dynamic gate adequacy; and module theorems: rendering-only and fresh-witness-family extensions are conservative at the frame's grade-free types, and new evidence may enable a construction but never silently. The book's running examples are written in the calculus, and all of the second part is checked in Lean as well.

Contents

1 Introduction **2**

2 The five roles **4**

3 Constructions **7**

4 Witnesses and scripts **8**

5 The book's examples **10**

6 The core calculus **11**

7 Reduction and definitional equality 17

8 Structural metatheory 18

9 Gate adequacy 23

10 Modules and extension theorems 26

11 Erasure and provenance 30

12 The book in the calculus 31

13 Relationship to existing work 39

14 Theorem status and limitations 40

15 Conclusion 42

A Complete declarative rule set 43

B Reduction contexts and equality 45

C Reference decision procedure 45

1 Introduction

The textbook *Artform Calculus* [1] teaches one discipline from its first chapter: every datum of a construction has a role, and the role decides what may depend on it. There are five roles. A FORM retains a carrier and its construction history. A PRESENTATION retains a selected order, seams, coordinates, cuts or sectors. A WITNESS retains evidence that a comparison or a move is allowed. A SCRIPT is a total action on the legal states of a presentation. A RENDERING is one chosen visible reading, and no rule consumes it [1, §1.9]. The book's chess example puts all five on one board: the squares with their adjacency are form, the addressing $a1$ to $h8$ is presentation, the certificate that a move is legal is a witness, the move is a script, and the printed diagram and an engine's evaluation are renderings.

The book claims more than a vocabulary. It says that the list is closed, that a sixth role would make a different calculus, and that one rule carries most of the weight: nothing may depend on a reading. This paper asks what it takes for those claims to be solid, and proves what it can. Four questions organize the answer.

(1) *Is the policy forced?* The book states the policy in prose. Four requirements that the book states determine it uniquely (Theorem 2.5).

(2) *Does checking each dependency suffice?* A policy is useful only if a construction can be checked one dependency at a time. That is so exactly for path-closed policies, and the book's policy is path closed (Theorems 2.6 and 3.2).

(3) *Do the guarantees hold?* Nothing outside the rendering depends on a reading; every dependency the policy gates crosses a gate whose record reaches the result; the frame changes only on the record; and a form never depends on its addressing except on the record. Each holds for chains of dependencies and for computed values (section 3).

(4) *Are there exactly five roles?* The policy tells nine of the ten pairs of roles apart. It cannot tell WITNESS from SCRIPT, and should not; their rules tell them apart, and the book's

counts of histories depend on the difference (section 4). A state, which the book declines to call a role, is witness data, so every datum has one of the five roles (section 2.2).

Sections 2 to 5 answer these questions for the role discipline alone. Every theorem there is checked in Lean 4 with Mathlib [3, 4], in the library `ArtformRoles` of the repository that accompanies the book [2]; table 5 lists the Lean names. The proofs use only the axioms `propext`, `Classical.choice` and `Quot.sound`.

The book keeps the rules of inference, which are Martin-Löf's [6], and the roles as two disciplines: the rules check that each value has its type, and only the roles say what a value may be used for [1, §1.11]. Sections 6 to 11 put the two into one judgement. The calculus $\mathsf{RATT}_{\mathrm{c}}$, a role-stratified artform type theory, is a universe-free dependent type theory whose judgement $\mathfrak{S}; \Gamma \vdash_{\rho} t : A \triangleright \varepsilon$ carries the role ρ of its result and the effect ε of every declaration it uses. An effect is a vector of grades, and its paths are exactly the dependency chains of section 3 (Lemma 8.1). A registered gate is the only rule that may introduce a gated dependency, and its contraction leaves a node that records the event which authorized it. The calculus, its metatheory and the examples of section 12 are checked in Lean in the same library, namespace `Artform.Roles.Calculus`. The mechanization corrected two side conditions of the signature rules and the scope of a few claims of an earlier version; section 14 lists every change.

1.1 Contributions

The paper contributes:

(i) the book's five roles with the book's readings, and a decision that makes them classify every datum: a state is witness data (section 2.2);

(ii) the generation of the policy by four levels, its uniqueness under the book's four requirements, and its path closure (Theorems 2.4 to 2.6);

(iii) constructions with graded dependencies, on which checking each direct dependency suffices exactly for path-closed policies, and the firewall, recorded mediation, frame stability and presentation independence for chains and for values (section 3);

(iv) the separation of all ten pairs of roles: nine by the policy, and WITNESS from SCRIPT by the count of histories (Theorem 4.5);

(v) Lean proofs of all of the above, and of the book's chess, realizer and walker examples (section 5 and table 5);

(vi) the calculus $\mathsf{RATT}_{\mathrm{c}}$ with principal min–max effects, graded dependent functions, stamped credentials, registered gates and private record nodes, and Lean proofs of substitution, preservation, strong normalization, confluence, canonical forms, decidable conversion and type checking, path mediation and static–dynamic gate adequacy;

(vii) module theorems that distinguish rendering vocabulary, fresh witness families and new inhabitants of old evidence families, at an explicit grade-free boundary; and

(viii) the book's frames, states, scripts, interpretation witnesses and renderings written in $\mathsf{RATT}_{\mathrm{c}}$, with their typings checked in Lean.

1.2 Relation to an earlier manuscript

This paper revises the manuscript *Traceable Gates for Artforms* (September 2026), whose calculus is the second part here. There the roles were written $\mathsf{O}, \mathsf{S}, \mathsf{W}, \mathsf{T}, \mathsf{R}$ and read as constructed objects, installed structure, evidence, transformations and renderings. Here they carry the book's

names and readings, and the policy is the same matrix. Three things are new: the first part, which justifies the five roles and is checked in Lean; the decision that a state is witness data; and the word *record* for what a gate contraction leaves, since the book uses *receipt* for a witness of a measured feature [1, Prop. 3.43].

2 The five roles

2.1 Roles

Table 1 gives the five roles with the book's readings [1, §1.9]. A role is a job that a datum does at a stage of a construction, not a kind of substance: the base cut of a wheel is a witness while it is offered to the constructor that points the wheel, and presentation once the pointed wheel carries it. The role of a datum is the role of the type at which it is judged at that stage, so the question a role answers is never what a thing is, but what it is doing now.

Role		What it retains	Typical examples
FORM	F	carrier and construction history	cells, tags, order, incidence, orientation
PRESENTATION	P	selected order, seams, coordinates, cuts, sectors	rod, grid view, wheel, coordinate system
WITNESS	W	evidence that a comparison or move is allowed, and the choices it certifies	alignment, contact, a measurement certificate, a supplied policy, a state
SCRIPT	S	total action on the legal states of a presentation	flatten, reflection, update, forgetful passage, readout
RENDERING	R	one chosen visible reading, consumed by no rule	drawing, table, sound, a value read off

Table 1: The five roles. The readings are the book's; the witness row adds the choices a witness certifies, among them a state (section 2.2).

Definition 2.1 (Roles)**.** The roles are $\mathsf{Role} = \{\mathsf{F}, \mathsf{P}, \mathsf{W}, \mathsf{S}, \mathsf{R}\}$, read as in table 1. Throughout, every role metavariable $\sigma, \tau, \kappa, \rho$, with or without an index, ranges over Role.

A *frame* is a typed specification of a carrier, its carried structure, the space of legal states, the allowed observations and the allowed moves; an *instance* is one legal state inhabiting a frame [1, Def. 1.13]. The carrier and its construction are form, the carried structure is presentation, the moves are scripts, and an observation is a script whose values are renderings. The frame does not change while a run proceeds; the instance does.

2.2 States

The book calls a state one side of the frame/instance split rather than a job, and declines to make it a sixth role. It also says which role a state's data have: when a role must be named for a state, it is a witness [1, §1.9]. Earlier editions said instead to name the presentation that the state inhabits; the reasons for the present answer follow.

A state is a choice of values at the positions of a frame, made legal by evidence. In the book's formation rule the state space of a presentation A is the tagged choice of a table with the evidence that it is legal, $\Sigma_s \mathsf{Legal}_A(s)$, and the legality is counted as a property: different proofs of the same legality claim do not make different states [1, §1.9]. Choices and evidence are the witness role's data. The book counts a supplied policy and a chosen batch as witnesses, and says of its process laboratory that evidence and choice are one role. So a state is witness data. It is not carried structure, since a law changes it while the frame stays fixed; and it is not a script, since it does not act. The book's connection, a witness family that became a state, keeps its role when it is gathered into one state [1, §5.9], and a derived state such as the squares one

side bears on is witness data that a rule may consult (section 5.1). The rule concerns the states of a frame, whose positions are fixed. A process that grows its carrier, as the book's growth does, has forms for its stages, and each step of growth is a gate into a form.

Principle 2.2 (Every datum has one of the five roles)**.** In a construction every datum is form, presentation, witness, script or rendering. The book's four useful words that are not roles have these homes:

state witness data: a legal choice of values;

policy witness data while it is supplied, presentation once a gate installs it (section 5.2);

reduct the result of a script that forgets part of a vocabulary;

observable a script whose values are renderings.

2.3 Grades and the policy

A dependency of one datum on another is used in one of three ways: in the ordinary way, through a gate that records what was assumed, or not at all. Let

$$\mathbb{G} = \{0 < 1 < \infty\}, \qquad p \oplus q = \min(p, q), \qquad p \otimes q = \max(p, q).$$

Grade 0 is ordinary use, grade 1 is use through a gate, and ∞ is no use. Two dependencies in sequence compose with $\otimes$; of two alternative routes, the better counts, by $\oplus$. Then $(\mathbb{G}, \oplus, \infty, \otimes, 0)$ is an idempotent commutative semiring.

The book states its policy in prose [1, §1.9]. Whatever has been built, presented, certified or done may be used freely downstream; a witness or a script may feed back into a presentation or a form only through a gate that records what was assumed; and nothing may depend on a reading, by any route through a third role. Table 2 is the policy as a matrix $D : \mathsf{Role} \times \mathsf{Role} \to \mathbb{G}$. Its entry $D(\sigma, \rho)$ is the least grade at which a datum of role ρ may depend on a datum of role σ. It is the matrix of the earlier manuscripts, under the book's names. The book's prose names every gated entry, including the gate from presentation to form, which its editions before October 2026 left implicit (section 2.4).

source / result	F	P	W	S	R
F	0	0	0	0	0
P	1	0	0	0	0
W	1	1	0	0	0
S	1	1	0	0	0
R	∞	∞	∞	∞	0

Table 2: The dependency policy D. Rows are the roles depended on, columns the roles that depend. A demand of ∞ means that only an absent dependency is accepted.

2.4 Four levels

Definition 2.3 (Levels)**.** The *level* of a role is

$$\ell(\mathsf{F}) = 0, \qquad \ell(\mathsf{P}) = 1, \qquad \ell(\mathsf{W}) = \ell(\mathsf{S}) = 2, \qquad \ell(\mathsf{R}) = 3,$$

read as the frame's carrier, the frame's carried structure, the instance, and the display. For any map $\lambda : \mathsf{Role} \to \mathbb{N}$ let

$$D_\lambda(\sigma, \rho) = \begin{cases} \infty & \text{if } \sigma = \mathsf{R} \neq \rho, \\ 0 & \text{otherwise, if } \lambda(\sigma) \leq \lambda(\rho), \\ 1 & \text{otherwise.} \end{cases}$$

Theorem 2.4 (The policy has four levels)**.** *$D = D_\ell$. A dependency is free from a lower or equal level, gated from a higher one, and absent out of a reading.*

The book gives four reasons for the policy, and each is a requirement on a level map λ.

(R1) *Nothing may depend on a reading, and a reading may be taken of anything*: $\lambda(\sigma) < \lambda(\mathsf{R})$ for every $\sigma \neq \mathsf{R}$.

(R2) *The frame does not change while a run proceeds*; a witness or a script reaches a form or a presentation only through a gate: $\lambda(\mathsf{F}), \lambda(\mathsf{P}) < \lambda(\mathsf{W}), \lambda(\mathsf{S})$.

(R3) *No addressing is part of the board*: a form does not depend on how it is addressed, so $\lambda(\mathsf{F}) < \lambda(\mathsf{P})$.

(R4) *A certificate may mention a move, and a guarded move takes its certificate*: the certificate that a capture is legal names the capture, and a guarded move is total on its admitted input, including the needed evidence [1, §1.9]. Neither direction is feedback, so $\lambda(\mathsf{W}) = \lambda(\mathsf{S})$.

Theorem 2.5 (The policy is forced)**.** *The levels ℓ satisfy* (R1)–(R4)*, and every map $\lambda : \mathsf{Role} \to \mathbb{N}$ satisfying* (R1)–(R4) *has $D_\lambda = D$.*

Proof. The requirements fix every comparison between the values of λ on two roles, and D_λ depends only on those comparisons and on the firewall clause. □

2.5 Path closure

A policy is *path closed* when

$$D(\sigma, \rho) \leq D(\sigma, \tau) \otimes D(\tau, \rho) \qquad \text{for all } \sigma, \tau, \rho. \tag{2.1}$$

The book's key idea says that no route through a third role recovers a forbidden dependency; path closure is that sentence for every pair of steps.

Theorem 2.6 (Path closure)**.** *The policy D is path closed, and $D(\sigma, \sigma) = 0$ for every role.*

For example, a script may freely produce evidence, but a dependency of a presentation on that evidence still has demand 1; and a witness built from a reading would carry the reading's demand $\boldsymbol{\infty}$ into everything built from the witness. Theorem 3.2 shows that path closure is exactly what makes one-step checking sound.

2.6 What the policy tells apart

Say that the policy *separates* two roles σ and τ when some role κ has $D(\sigma, \kappa) \neq D(\tau, \kappa)$ or $D(\kappa, \sigma) \neq D(\kappa, \tau)$: some datum may depend on one of them in a way it may not depend on the other, or one of them may depend on something in a way the other may not.

Theorem 2.7 (Policy separation)**.** *The policy separates σ and τ if and only if $\sigma \neq \tau$ and $\{\sigma, \tau\} \neq \{\mathsf{W}, \mathsf{S}\}$. The rows of W and S are equal, and so are their columns.*

Table 3 names, for each separated pair, a role on which the two differ and the guarantee of section 3 that the difference carries. Merging the two roles of a pair would either forbid a dependency the book makes or permit one the guarantee excludes.

Pair	Where the demands differ	Guarantee
F, P	$D(\mathsf{P},\mathsf{F}) = 1$, $D(\mathsf{F},\mathsf{F}) = 0$	presentation independence (Theorem 3.10)
F, W; F, S	$D(\mathsf{W},\mathsf{F}) = D(\mathsf{S},\mathsf{F}) = 1$, $D(\mathsf{F},\mathsf{F}) = 0$	frame stability for forms (Theorem 3.9)
P, W; P, S	$D(\mathsf{W},\mathsf{P}) = D(\mathsf{S},\mathsf{P}) = 1$, $D(\mathsf{P},\mathsf{P}) = 0$	frame stability for presentations (Theorem 3.9)
σ, R for $\sigma \neq \mathsf{R}$	$D(\mathsf{R},\sigma) = \infty$, $D(\sigma,\sigma) = 0$	the firewall (Corollary 3.3 and Theorem 3.8)
W, S	nowhere	the count of histories (Theorem 4.5)

Table 3: How each pair of roles is told apart.

3 Constructions

3.1 Graded dependencies

Definition 3.1 (Construction)**.** A *construction* on a set V of items gives each item v a role $\mathsf{role}(v)$ and each pair of items u, v a grade $c(u, v) \in \mathbb{G}$: the grade at which v depends directly on u, with ∞ for no dependency. It is *valid* for a policy D when

$$D(\mathsf{role}(u), \mathsf{role}(v)) \leq c(u, v) \qquad \text{for all } u, v \in V.$$

For a grade g, a *g-chain* from u to v is a sequence $u = v_0, v_1, \ldots, v_k = v$ with $c(v_{i-1}, v_i) \leq g$ for every i; the empty chain has $k = 0$. A 0-chain uses ordinary dependencies only; a 1-chain is a *live* chain.

The derivations of section 6 give constructions: the items are the declarations and variables a derivation uses and its result, and the grades are the coefficients of its effect equations. Lemma 8.1 shows that the effect of a derivation is the best grade over its chains.

Theorem 3.2 (Local to global)**.** *Let D be a policy with $D(\sigma, \sigma) = 0$ for every role.*

(i) If D is path closed, then in every construction valid for D, a g-chain from u to v exists only if $D(\mathsf{role}(u), \mathsf{role}(v)) \leq g$.

(ii) Conversely, if (i) *holds for every construction on three items, then D is path closed.*

Proof. (i) By induction on the chain: the empty chain needs $D(\sigma, \sigma) = 0$, and a chain extended by one dependency needs one instance of (2.1). (ii) For roles σ, τ, ρ, give three items these roles, let the first two dependencies have grades $D(\sigma, \tau)$ and $D(\tau, \rho)$, and let every other grade be ∞. The construction is valid and has a $D(\sigma, \tau) \otimes D(\tau, \rho)$-chain from the first item to the third. □

From now on a construction is valid for the book's policy D.

Corollary 3.3 (Firewall)**.** *No live chain leads from a rendering to a datum of another role.*

Corollary 3.4 (Mediation)**.** *If $D(\mathsf{role}(u), \mathsf{role}(v)) = 1$, no chain of ordinary dependencies leads from u to v.*

Theorem 3.5 (Records)**.** *If $D(\mathsf{role}(u), \mathsf{role}(v)) = 1$ and a live chain leads from u to v, then the chain crosses a gate: there are items w, w' with $c(w, w') = 1$, a live chain from u to w, and a live chain from w' to v. The record the gate leaves at w' reaches v.*

Proof. A live chain either has only ordinary dependencies, which Corollary 3.4 excludes, or contains a dependency of grade 1; split it there. □

3.2 Values

The statements so far are about chains. The guarantees the book wants are about values: what a reading changes, and what a run may change.

Definition 3.6 (Evaluation)**.** An *evaluation* of a construction with values in a set X gives each item v a map $f_v : X^V \to X$ that depends only on the values of the items u with $c(u, v) \leq 1$. A valuation $x \in X^V$ is *consistent* with it when $x_v = f_v(x)$ for every v. A construction is *acyclic* when some rank $r : V \to \mathbb{N}$ has $r(u) < r(v)$ whenever $c(u, v) \leq 1$.

Lemma 3.7 (Determination)**.** *Let a construction be acyclic, let $U \subseteq V$, and let two evaluations agree on the items of U. If x and y are consistent with the two evaluations and $x_u = y_u$ whenever $c(u, v) \leq 1$ for some $v \in U$ and $u \notin U$, then x and y agree on U.*

Proof. By induction on the rank of the item. □

Theorem 3.8 (Firewall for values)**.** *Replacing every renderer of an acyclic construction, and nothing else, changes no value outside the rendering: if two evaluations agree on every item that is not a rendering, consistent valuations agree there.*

Theorem 3.9 (Frame stability)**.** *Let two evaluations of an acyclic construction agree on its forms and presentations, and let consistent valuations x and y agree on the source of every gate into a form or a presentation. Then x and y agree on every form and presentation. The frame changes only through recorded gates.*

Theorem 3.10 (Presentation independence)**.** *Let two evaluations of an acyclic construction agree on its forms, and let consistent valuations agree on the source of every gate into a form. Then they agree on every form. In particular a form depends on how it is addressed only through recorded gates.*

Each of the three is Lemma 3.7 for one set U: the items that are not renderings, the frame's items, and the forms. Validity says that every live dependency entering U from outside it is a gate, or, for the firewall, that there is none.

3.3 Derived is not displayed

Two values may be computed from one state by explicit computation and still have different roles. The book's example is a chess position: the squares one side bears on form a table with a value on every square, and a rule may consult it; an engine's evaluation is one number, and no rule of chess consults it [1, §1.9]. In a construction the difference is the role of the item, which is the role of the type at which it is judged: the codomain decides, not the amount of computation. The first table is witness data, a derived state; the evaluation is a rendering, and Corollary 3.3 forbids every live chain from it to a certificate (section 5.1).

The same test settles the book's other pairs: curvature, a value on every face, is a derived state, while a holonomy read off one loop is a reading; a conserved charge is reported, while the divergence clause on every site is a constraint between states.

4 Witnesses and scripts

4.1 What the policy cannot do

Theorem 2.7 leaves one pair: witnesses and scripts have the same row and the same column of the policy. Requirement (R4) explains why, and the book needs both directions. Witnesses depend on scripts: the certificate of a capture names the capture, the reversibility of a shear is a property of the shear, and the evidence of a realizer says that every batch it selects keeps the

invariants. Scripts depend on witnesses: a guarded move takes its certificate, and the book's spatial reading of a measurement witness is a script built from that witness. Making either direction a gate would demand a record where the book has none.

Their rules tell them apart. A script acts: it has a source and a target, composes at the presentation in between, and has an action on legal states. A witness admits or chooses: it is consumed by a guard or a gate, and when it admits an application, which certificate it is does not matter. The book makes the difference count [1, §7.4].

4.2 Histories

A law on a finite set X of labels gives, for labels y and z, the finite set $\mathsf{App}(y, z)$ of its applications from y to z, which are script data, and for each application a the finite set $\mathsf{Ev}(a)$ of its certificates, which are witness data. An application is *admitted* when it has a certificate. A history of n steps is a list of n admitted applications, each starting where the previous one ended.

The book weights histories to obtain its tallies. With weight 1 a history counts once; with nonnegative weights it gives the probabilities of a stochastic realizer; with the coefficients of a coherent law it gives the quantum tallies, and interference is cancellation between two histories with one endpoint. In each case the book counts applications and not proofs: if each proof of an application's legality counted as a value, one move would be two histories and its coefficient would be counted twice [1, §7.4].

Definition 4.1 (Three counts)**.** Fix a start $x_0 \in X$. For $n \geq 0$ and $z \in X$ let $h_0(z) = p_0(z) = t_0(z) = [z = x_0]$ and

$$h_{n+1}(z) = \sum_{y \in X} \sum_{a \in \mathsf{App}(y,z)} [\mathsf{Ev}(a) \neq \varnothing]\, h_n(y),$$
$$p_{n+1}(z) = \sum_{y \in X} \sum_{a \in \mathsf{App}(y,z)} |\mathsf{Ev}(a)|\, p_n(y),$$
$$t_{n+1}(z) = \sum_{y \in X} \big[\exists a \in \mathsf{App}(y, z).\ \mathsf{Ev}(a) \neq \varnothing\big]\, t_n(y).$$

The *histories* $h_n(z)$ count lists of admitted applications, the book's count. The *proof lists* $p_n(z)$ count every certificate as a value. The *trajectories* $t_n(z)$ count sequences of labels joined by admitted applications, so that applications count only by their endpoints.

Proposition 4.2. *If every application has at most one certificate, then $p_n = h_n$. If between any two labels at most one application is admitted, then $h_n = t_n$.*

Example 4.3 (Two corners)**.** Let $X = \{0, 1\}$. From either corner the token may step ahead or behind, and both steps reach the other corner. Stepping ahead has two certificates and stepping behind has one. After one step from 0,

$$p_1(1) = 3, \qquad h_1(1) = 2, \qquad t_1(1) = 1.$$

4.3 Five roles

The three counts differ in how they count two kinds of data. Data can be counted as *values*, each datum once, or as a *property*, once if there is one. The book counts applications, script data, as values and certificates, witness data, as a property.

Definition 4.4 (Counting conventions)**.** Let $h_n^{m,m'}$ count lists of applications with applications counted in mode m and certificates in mode m', where a mode is *value* or *property*. A *counting convention* assigns one mode to each role; it counts *uniformly by role* when applications and certificates are counted in the modes of their roles.

The three counts are three conventions: $h^{\mathrm{val,prop}} = h$, $h^{\mathrm{val,val}} = p$, and $h^{\mathrm{prop},m'} = t$ for both modes m', since applications counted as a property count only their endpoints.

Theorem 4.5 (Separation)**.** *Every two distinct roles are told apart.*

(i) If $\{\sigma, \tau\} \neq \{\mathsf{W}, \mathsf{S}\}$, the policy separates σ and τ, and table 3 names the guarantee the difference carries.

(ii) Under counting that is uniform by role, witnesses and scripts are separated by the count of histories: if applications and certificates have one role, then one step from 0 to 1 on the law of Example 4.3 is counted 3 or 1, never the book's 2. A convention that counts as the book does therefore puts applications and certificates in different roles, and the book's convention, scripts as values and witnesses as properties, does.

Proof. (i) is Theorem 2.7. (ii) One role gives applications and certificates one mode, so the count is p or t by the identities above, and Example 4.3 gives 3 and 1. □

Remark 4.6 (Without uniform counting). The restriction in (ii) is needed. Identifying S with W changes no entry of the policy, and the counts of Definition 4.1 do not mention roles; a discipline may keep one role with two families, one counted as values and one used only through inhabitation, and count as the book does. What (ii) shows is that such a discipline counts the data of one role in two ways. The counts motivate two roles for two jobs; they are not a proof that every calculus needs exactly five roles.

With Principle 2.2 this answers the fourth question within its premises. Among the policies generated by levels, (R1)–(R4) force four levels (Theorem 2.5). Each level holds one role, except the instance level, which holds the two roles that act and that admit or choose, and under uniform counting these are two roles. A sixth role would need a fifth level, and so a requirement the book does not state, or a third job at the instance level that is neither acting nor admitting nor choosing. This is the book's vocabulary made precise, not a minimality theorem for other disciplines.

5 The book's examples

5.1 One capture

The book's §1.9 follows one capture, $N \times e5$, with the five roles. Table 4 lists its items as a construction. The board's squares with their adjacency and the step shape of each piece are form; the addressing is presentation; the positions before and after the capture, the squares White bears on, and the certificate that the capture is legal are witness data; the capture is a script; the game score is a form that keeps the moves; the engine's evaluation and the printed diagram are renderings. The score keeps the capture through a gate, and every other dependency is ordinary.

Proposition 5.1. *The capture of table 4 is valid. It becomes invalid if the certificate depends on the evaluation, or if the score depends on the capture without a gate. No live chain leads from the evaluation to the certificate or to the position after the capture.*

5.2 A realizer

A law says which moves are allowed and chooses none; a realizer is a policy together with evidence that every batch it selects keeps the law's invariants [1, Def. 5.10]. The policy is one datum at two stages. While it is supplied it is witness data; once a realizer carries it, it is presentation. As a construction: the law is a form, the supplied policy a witness that depends on the law, and the realizer a presentation that depends on the law and, through a gate, on

Item	Role	Depends on
board	F	—
addressing	P	board
position	W	board, addressing
squares White bears on	W	board, position
certificate of $N{\times}e5$	W	position, squares White bears on, the capture
capture $N{\times}e5$	S	—
position after	W	position, capture, certificate
game score	F	capture, through a gate
evaluation	R	position
diagram	R	position, addressing

Table 4: One capture as a construction.

the supplied policy. The construction is valid, no chain of ordinary dependencies leads from the supplied policy to the realizer, and a live chain does, across the installation gate (Corollary 3.4 and Theorem 3.5).

5.3 The walker

The book's walker moves a token round the four corners of a square, one step ahead or behind at every tick, and each step has one certificate: that the token is at the corner it leaves [1, Def. 5.9]. From v_0 there are four histories of two ticks; two end at v_0 and two at v_2. Each step has one certificate and leads to a corner of its own, so by Proposition 4.2 the three counts of Definition 4.1 agree on the square for every number of ticks. They come apart only when a step has two certificates, or two steps share an endpoint, as on the two corners of Example 4.3.

6 The core calculus

This section and the five after it give $\mathsf{RATT}_{\mathrm{c}}$, the dependent calculus whose judgements carry the roles. Its policy is the matrix D of table 2, and its grades are those of section 2.3. The roles F and P, whose data make up the frame, are called *protected*: a gated dependency can only end in them.

6.1 Effects

For a finite set of atoms Z, an *effect* is a vector $\varepsilon : Z \to \mathbb{G}$ with default value $\boldsymbol{\infty}$. Define

$$(\varepsilon \oplus \eta)(z) = \varepsilon(z) \oplus \eta(z),$$
$$(q \otimes \varepsilon)(z) = q \otimes \varepsilon(z),$$
$$\mathbf{e}_z(z) = 0, \qquad \mathbf{e}_z(y) = \boldsymbol{\infty} \quad (y \neq z).$$

Restriction deleting x is written $\varepsilon \setminus x$. Closed literals and meta-level finite indices have no declaration atom and therefore introduce no dependency. Let $\mathsf{At}(\mathfrak{S}, \Gamma)$ contain the public declarations of $\mathfrak{S}$ and the variables of Γ, each with its declared role.

Definition 6.1 (Policy-valid effect)**.** An effect ε at result role ρ is policy valid, written $\mathsf{valid}_\rho(\varepsilon)$, when

$$D(\mathsf{role}(z), \rho) \leq \varepsilon(z) \qquad (z \in \mathsf{At}(\mathfrak{S}, \Gamma)).$$

Thus demand 1 requires every live path from z to contain a gate, and demand $\boldsymbol{\infty}$ requires that no live path remain.

A value computed from an artform has the role of the type at which it is judged: the codomain decides, and there is no role-free escape hatch for derived data.

Result	Lean name
semiring of grades	`Grade.semiring_laws`
Theorem 2.4	`demand_eq_demandOf_level`
Theorem 2.5	`requirements_level`, `demandOf_eq_demand`
Theorem 2.6	`demand_pathClosed`, `demand_self`
Theorem 2.7	`policySeparates_iff`, `witness_script_same_policy`
Theorem 3.2	`Construction.Valid.reach`, `Construction.pathClosed_of_reach`
Corollaries 3.3 and 3.4	`Construction.no_reach_of_rendering`, `Construction.no_free_reach`
Theorem 3.5	`Construction.crosses_gate`
Lemma 3.7	`Construction.Evaluation.eq_of_agree`
Theorems 3.8 to 3.10	`Construction.firewall`, `Construction.frame_stable`, `Construction.form_stable`
Proposition 4.2	`Law.proofs_eq_histories`, `Law.histories_eq_trajectories`
Example 4.3	`twoCorners_counts`
Definition 4.4	`Law.countWith_book`, `Law.countWith_values`, `Law.countWith_properties`
Theorem 4.5(ii)	`merged_roles_miscount`, `book_count_separates`, `book_convention_counts`
remark 4.6	`collapseRole_keeps_policy`
Proposition 5.1	`chess_valid`, `chess_readback_invalid`, `chess_silent_score_invalid`, `chess_evaluation_terminal`
Section 5.2	`realizer_valid`, `realizer_installation_recorded`
Section 5.3	`walker_two_ticks`, `walker_counts_agree`

Table 5: The results of sections 2 to 5 in the Lean library `ArtformRoles`, namespace `Artform.Roles`.

6.2 The universe-free boundary

The common Artforms notation uses $\mathfrak{S}$ for a declaration signature and Σ, Π for dependent type formers. The role and grade annotations in this paper are part of their checked interfaces: pairs remain homogeneous in role, and type remains a judgmental sort. Upright sans-serif names share the corpus spelling; no ambient datum thereby becomes an internal declaration.

RATT_c has dependent functions and pairs but no internal universe. Types are classified by a meta-level judgment rather than packaged as terms. This choice removes the standard counterexample $\Sigma_{X:\mathsf{Type}}X$, in which a fresh type and one constructor form a new inhabitant of an old type. Role-indexed universes are an important extension, but including them before their elimination rules are proved noninterfering would weaken the main theorem.

The core also excludes arbitrary user rewrite rules, quotient computation, and extensions of script equality. Strictly positive inductive families and certified recursion may be added through a host-kernel interface, but are not needed to state or prove the gate metatheory. Forced algebraic-value extension from the Pattern Geometry manuscript remains a separate staged problem [5].

6.3 Syntactic categories

The calculus is an LF-style dependent type theory: type is a judgmental sort, not an internal term. A role is an index of a declaration or judgment, not part of the type itself. Here and in every rule below, $\sigma, \rho \in \mathsf{Role}$. We therefore write $x :_\sigma A$ for a declaration at source role σ, and $\vdash_\rho$ for a formation or typing judgment at result role ρ. The subscripted colon is also used in binders

and telescopes; $\Rightarrow_\rho$ marks the result role of a family or gate declaration. The unindexed colon in $\vdash_\rho t : A$ retains its ordinary meaning: t has type A. Grades decorate dependent function spaces.

The five-element set Role is the global range from which role indices are drawn, but the role attached to a particular well-formed type is not an arbitrary choice from that set. We call ρ an *admissible outer role* of A when a formation judgment $\mathfrak{S}; \Gamma \vdash_\rho A\ \ \mathsf{type}$ is derivable. The formation rules below make this outer role unique. Parameters and proper subexpressions of A may nevertheless carry different roles.

$$\begin{aligned} A, B &::= a(\vec{t}) \mid \mathsf{Event} \mid \mathsf{Origin}_A(e,t) \mid \mathsf{Stamped}_\sigma(A) \mid \Pi^q(x :_\sigma A).B \mid \Sigma(x :_\rho A).B, \\ t, u &::= x \mid c \mid \lambda^q(x :_\sigma A).t \mid t\,u \mid (t,u) \mid \mathsf{fst}(t) \mid \mathsf{snd}(t) \\ &\quad \mid \mathsf{stamp}(t,e,o) \mid \mathsf{gate}_g(\vec{d}; s) \mid \mathsf{node}_g(\vec{d}; t, e). \end{aligned}$$

Here $q \in \mathbb{G}$, a is a declared type-family symbol, and c is a declared public constant. The last form is internal syntax: users cannot write node_g, but reduction can produce it. Dependent sums are homogeneous in role. Cross-role packaging is provided only by the sealed $\mathsf{Stamped}$ former and consumed only by a gate.

An expression is *public* when it contains no private node_g form. Write $\mathsf{gradefree}(A)$ when every Π-annotation occurring anywhere in A is 0. This is a syntactic observation boundary, not a claim that grade-zero functions are first order: it excludes only types that can retain a forbidden dependency behind a positive or absent latent function edge.

For exact attestation, let $\mathsf{wdata}(A)$ be generated by

$$\frac{a : (\Delta) \Rightarrow_{\mathsf{W}} \mathsf{type}}{\mathsf{wdata}(a(\vec{t}))} \qquad \frac{\mathsf{wdata}(A) \quad \mathsf{wdata}(B)}{\mathsf{wdata}(\Sigma(x :_{\mathsf{W}} A).B)}. \tag{WitnessData}$$

Thus witness data consists of witness-family applications and finite homogeneous dependent sums of them. Event, origin, stamp, function, rendering, and non-witness heads are excluded.

A context is

$$\Gamma ::= \cdot \mid \Gamma, x :_\rho A.$$

The principal judgments are

$$\mathfrak{S}\ \mathsf{wf}, \qquad \mathfrak{S} \vdash \Gamma\ \mathsf{ctx},$$
$$\mathfrak{S}; \Gamma \vdash_\rho A\ \ \mathsf{type} \triangleright \varepsilon, \qquad \mathfrak{S}; \Gamma \vdash_\rho t : A \triangleright \varepsilon.$$

Effects record free variables and public declaration atoms. Every displayed formation or typing rule has the side condition that its conclusion effect is policy valid. We write a superscript ✓ on a premise when that side condition has already been discharged.

6.4 Signatures

A signature is a finite ordered list of the following entries.

Family $a : (\Delta) \Rightarrow_\rho \mathsf{type}$, where Δ is an already well-formed telescope.

Primitive $c :_\rho A$, where A is closed and policy valid. Every Π-grade in the type of a primitive is 0, and its ultimate result head is a declared family symbol. Thus a primitive cannot merely assert that it mediates an argument, introduce an opaque pair, or manufacture an unowned credential.

Certificate bundle
For closed $\vdash_\sigma A\ \ \mathsf{type}$ with $\sigma \neq \mathsf{R}$, no nonzero Π-grade, and ordinary result head, a bundle

$$d = (a_d, e_d, o_d) : A$$

introduces fresh public constants

$$a_d :_\sigma A, \qquad e_d :_{\mathsf{W}} \mathsf{Event}, \qquad o_d :_{\mathsf{W}} \mathsf{Origin}_A(e_d, a_d).$$

The three names have the same module owner.

Exact attestation

For a closed public term

$$\vdash_{\mathsf{W}} t : A, \qquad \mathsf{gradefree}(A), \qquad \mathsf{wdata}(A),$$

an entry $q@t = (e_q, o_q) : A$ introduces only

$$e_q :_{\mathsf{W}} \mathsf{Event}, \qquad o_q :_{\mathsf{W}} \mathsf{Origin}_A(e_q, t).$$

The two fresh names have the same module owner. The payload is the exact already constructed term t, not a new constant.

Gate

A gate entry

$$g : \Delta; \mathsf{Stamped}_\sigma(A) \Longrightarrow_\rho B \quad [\mathsf{node}_g]$$

requires $\rho \in \{\mathsf{F}, \mathsf{P}\}$, $D(\sigma, \rho) \leq 1$, and:

(G1) Δ is an ordinary parameter telescope whose declaration roles have demand 0 into ρ;

(G2) $\Delta \vdash_\sigma A$ type is derivable;

(G3) $\Delta \vdash_\rho B$ type is derivable, and B does not depend on the stamped payload, and contains no nonzero Π-grade; and

(G4) node_g is a fresh private constructor with ordinary parameter edges and grade-1 payload and event edges.

The public gate has the graded type

$$\Pi^0(\vec{x} : \Delta).\, \Pi^1(s :_{\mathsf{W}} \mathsf{Stamped}_\sigma(A)).\, B.$$

All names are globally qualified by their module. There is no declaration form for a custom reduction rule, no public declaration of a positive Π-grade, no positive grade in a gate's result type, no attestation of an open or non-witness-data term, and no eliminator from $\mathsf{Stamped}$. These negative clauses are part of signature well-formedness, not optional module advice.

6.5 Type formation and effects

The family rule uses ordinary edges:

$$\frac{a : (x_1 :_{\rho_1} A_1, \ldots, x_n :_{\rho_n} A_n) \Rightarrow_\rho \mathsf{type} \in \mathfrak{S} \quad \mathfrak{S}; \Gamma \vdash_{\rho_i} t_i : A_i[\vec{t}_{<i}/\vec{x}_{<i}] \triangleright \varepsilon_i}{\mathfrak{S}; \Gamma \vdash_\rho a(\vec{t}) \;\; \mathsf{type} \triangleright \mathbf{e}_a \oplus \bigoplus_i \varepsilon_i}. \tag{Ty-Fam}$$

The atom a makes a newly declared family visible even when it has no arguments. The built-in event type has no public declaration atom:

$$\frac{}{\mathfrak{S}; \Gamma \vdash_{\mathsf{W}} \mathsf{Event} \;\; \mathsf{type} \triangleright \boldsymbol{\infty}}. \tag{Ty-Event}$$

Here and below $\boldsymbol{\infty}$ also denotes the everywhere-$\boldsymbol{\infty}$ vector.

Origin and stamping are type formers at the witness role:

$$\frac{\mathfrak{S}; \Gamma \vdash_\sigma A \;\; \mathsf{type} \triangleright \alpha \quad \mathfrak{S}; \Gamma \vdash_{\mathsf{W}} e : \mathsf{Event} \triangleright \eta \quad \mathfrak{S}; \Gamma \vdash_\sigma t : A \triangleright \theta}{\mathfrak{S}; \Gamma \vdash_{\mathsf{W}} \mathsf{Origin}_A(e, t) \;\; \mathsf{type} \triangleright \alpha \oplus \eta \oplus \theta} \tag{Ty-Origin}$$

and

$$\frac{\mathfrak{S};\Gamma \vdash_\sigma A \;\; \mathsf{type} \triangleright \alpha \quad \sigma \neq \mathsf{R}}{\mathfrak{S};\Gamma \vdash_{\mathsf{W}} \mathsf{Stamped}_\sigma(A) \;\; \mathsf{type} \triangleright \alpha}. \tag{Ty-Stamp}$$

The grade on a dependent function is both a reduction-stable lower bound on its operational argument use and the edge by which its domain type enters the function type. The explicit premise $D(\sigma, \rho) \leq q$ checks the role of the binder itself, even when its domain has no public family atom (as for Event):

$$\frac{\mathfrak{S};\Gamma \vdash_\sigma A \;\; \mathsf{type} \triangleright \alpha \quad \mathfrak{S};\Gamma, x :_\sigma A \vdash_\rho B \;\; \mathsf{type} \triangleright \beta \quad q \in \mathbb{G} \quad D(\sigma,\rho) \leq q}{\mathfrak{S};\Gamma \vdash_\rho \Pi^q(x :_\sigma A).B \;\; \mathsf{type} \triangleright (q \otimes \alpha) \oplus (\beta \setminus x)}. \tag{Ty-Pi}$$

The premise for B separately checks any occurrence of x in the codomain. In particular, a gate result type cannot hide a witness-dependent index; gate results depend only on their ordinary telescope.

Dependent sums are homogeneous and have ordinary component edges:

$$\frac{\mathfrak{S};\Gamma \vdash_\rho A \;\; \mathsf{type} \triangleright \alpha \quad \mathfrak{S};\Gamma, x :_\rho A \vdash_\rho B \;\; \mathsf{type} \triangleright \beta}{\mathfrak{S};\Gamma \vdash_\rho \Sigma(x :_\rho A).B \;\; \mathsf{type} \triangleright \alpha \oplus (\beta \setminus x)}. \tag{Ty-Sigma}$$

Lemma 6.2 (Role functionality)**.** *For a fixed well-formed signature $\mathfrak{S}$ and context Γ, if*

$$\mathfrak{S};\Gamma \vdash_\rho A \;\; \mathsf{type} \triangleright \alpha \qquad \textit{and} \qquad \mathfrak{S};\Gamma \vdash_{\rho'} A \;\; \mathsf{type} \triangleright \alpha',$$

then $\rho = \rho'$. Hence every well-formed type has exactly one admissible outer role. In a declaration $x :_\sigma A$ or a term judgment $\vdash_\rho t : A$, the displayed role must be this unique outer role of A; it is not independently chosen from Role*.*

Proof. By inversion on the two formation derivations. For a family application, the unique declaration of its head symbol fixes the result role. The Event, Origin, and Stamped formers have the fixed result role W. A dependent function has the outer role of its codomain, so the claim follows inductively from the two derivations of that codomain. A dependent sum is homogeneous and its displayed component role is also its outer role. These are all type formers. Policy acceptance can reject a raw formation derivation but cannot change its role. □

6.6 Terms and principal effects

Variables and public constants introduce unit effects:

$$\frac{x :_\rho A \in \Gamma}{\mathfrak{S};\Gamma \vdash_\rho x : A \triangleright \mathbf{e}_x} \tag{Var}$$

$$\frac{c :_\rho A \in \mathfrak{S}}{\mathfrak{S};\Gamma \vdash_\rho c : A \triangleright \mathbf{e}_c}. \tag{Const}$$

For abstraction, the declared grade is a lower bound on the principal path effect of the bound parameter in the body.

$$\frac{\mathfrak{S};\Gamma \vdash_\sigma A \;\; \mathsf{type} \quad \mathfrak{S};\Gamma, x :_\sigma A \vdash_\rho t : B \triangleright \theta \quad q \leq \theta(x)}{\mathfrak{S};\Gamma \vdash_\rho \lambda^q(x :_\sigma A).t : \Pi^q(x :_\sigma A).B \triangleright \theta \setminus x}. \tag{Lam}$$

Application composes the argument paths with the latent grade:

$$\frac{\mathfrak{S};\Gamma \vdash_\rho f : \Pi^q(x :_\sigma A).B \triangleright \phi \quad \mathfrak{S};\Gamma \vdash_\sigma u : A \triangleright \eta}{\mathfrak{S};\Gamma \vdash_\rho f\, u : B[u/x] \triangleright \phi \oplus (q \otimes \eta)}. \tag{App}$$

For source elaboration, an omitted grade is chosen maximally and exactly as $q = \theta(x)$. An explicit smaller grade is a conservative promise: it may report more dependence than the current

body exhibits, but never less. This slack is required by subject reduction. For example, reducing $\lambda^0 x.\mathsf{fst}(c, x)$ to $\lambda^0 x.c$ must not change its function type. The binder-policy premise in Ty-Pi prevents such slack from turning a demand-1 or demand-$\boldsymbol{\infty}$ role crossing into a grade-zero interface.

Thus a maximally elaborated unused parameter receives grade $\boldsymbol{\infty}$, an ordinary use receives grade 0, and a parameter used only beneath gates receives grade 1. Every term judgment also requires a prior accepted formation derivation for its displayed type; this convention is made explicit in appendix A.

Pairing and projections are ordinary:

$$\frac{\mathfrak{S};\Gamma \vdash_\rho t : A \triangleright \theta \quad \mathfrak{S};\Gamma \vdash_\rho u : B[t/x] \triangleright \eta}{\mathfrak{S};\Gamma \vdash_\rho (t,u) : \Sigma(x :_\rho A).B \triangleright \theta \oplus \eta}, \tag{Pair}$$

$$\frac{\mathfrak{S};\Gamma \vdash_\rho p : \Sigma(x :_\rho A).B \triangleright \pi}{\mathfrak{S};\Gamma \vdash_\rho \mathsf{fst}(p) : A \triangleright \pi} \tag{Fst}$$

$$\frac{\mathfrak{S};\Gamma \vdash_\rho p : \Sigma(x :_\rho A).B \triangleright \pi}{\mathfrak{S};\Gamma \vdash_\rho \mathsf{snd}(p) : B[\mathsf{fst}(p)/x] \triangleright \pi}. \tag{Snd}$$

A stamp is a sealed triple. It has introduction but no user-accessible projection or recursor:

$$\frac{\mathfrak{S};\Gamma \vdash_\sigma t : A \triangleright \theta \quad \mathfrak{S};\Gamma \vdash_{\mathsf{W}} e : \mathsf{Event} \triangleright \eta \quad \mathfrak{S};\Gamma \vdash_{\mathsf{W}} o : \mathsf{Origin}_A(e,t) \triangleright \omega}{\mathfrak{S};\Gamma \vdash_{\mathsf{W}} \mathsf{stamp}(t,e,o) : \mathsf{Stamped}_\sigma(A) \triangleright \theta \oplus \eta \oplus \omega}. \tag{Stamp}$$

For a registered gate $g : \Delta; \mathsf{Stamped}_\sigma(A) \Rightarrow_\rho B$, write δ for the ordinary parameter effect and η for the stamp effect. Its public and private rules are

$$\frac{\mathfrak{S};\Gamma \vdash \vec{d} : \Delta \triangleright \delta \quad \mathfrak{S};\Gamma \vdash_{\mathsf{W}} s : \mathsf{Stamped}_\sigma(A[\vec{d}]) \triangleright \eta}{\mathfrak{S};\Gamma \vdash_\rho \mathsf{gate}_g(\vec{d}; s) : B[\vec{d}] \triangleright \mathbf{e}_g \oplus \delta \oplus (1 \otimes \eta)} \tag{Gate}$$

and

$$\frac{\mathfrak{S};\Gamma \vdash \vec{d} : \Delta \triangleright \delta \quad \mathfrak{S};\Gamma \vdash_\sigma t : A[\vec{d}] \triangleright \theta \quad \mathfrak{S};\Gamma \vdash_{\mathsf{W}} e : \mathsf{Event} \triangleright \eta}{\mathfrak{S};\Gamma \vdash_\rho \mathsf{node}_g(\vec{d}; t, e) : B[\vec{d}] \triangleright \mathbf{e}_g \oplus \delta \oplus (1 \otimes (\theta \oplus \eta))}. \tag{Node}$$

Only the reduction relation may invoke Node. The owner atom remains g; the private constructor does not create a second public capability.

Finally, conversion changes only the displayed type:

$$\frac{\mathfrak{S};\Gamma \vdash_\rho t : A \triangleright \theta \quad A \equiv_\rho B}{\mathfrak{S};\Gamma \vdash_\rho t : B \triangleright \theta}. \tag{Conv}$$

Example 6.3 (A higher-order gate wrapper)**.** Fix a registered gate $g : \Delta; \mathsf{Stamped}_{\mathsf{W}}(A) \Rightarrow_{\mathsf{F}} B$ and closed ordinary parameters $\vec{d} : \Delta$. In the body

$$\mathsf{gate}_g(\vec{d}; c)$$

the bound credential c has effect component 1. Consequently

$$G \;=\; \lambda^1(c :_{\mathsf{W}} \mathsf{Stamped}_{\mathsf{W}}(A)).\mathsf{gate}_g(\vec{d}; c)$$

has type $\Pi^1(c :_{\mathsf{W}} \mathsf{Stamped}_{\mathsf{W}}(A)).B$. For a certificate bundle (a_d, e_d, o_d), application reduces as

$$G\,\mathsf{stamp}(a_d, e_d, o_d) \longrightarrow \mathsf{gate}_g(\vec{d}; \mathsf{stamp}(a_d, e_d, o_d)) \longrightarrow \mathsf{node}_g(\vec{d}; a_d, e_d).$$

Thus the latent grade permits higher-order abstraction without replacing the record-producing gate that justifies it.

All side conditions, simultaneous telescope rules, and context/signature formation rules are collected without abbreviation in appendix A.

7 Reduction and definitional equality

7.1 One-step reduction

Capture-avoiding substitution is written $t[u/x]$. The primitive redexes are

$$(\lambda^q(x :_\sigma A).t)\, u \longrightarrow t[u/x], \tag{β_q}$$
$$\mathsf{fst}(t, u) \longrightarrow t, \tag{π_1}$$
$$\mathsf{snd}(t, u) \longrightarrow u, \tag{π_2}$$
$$\mathsf{gate}_g(\vec{d}; \mathsf{stamp}(t, e, o)) \longrightarrow \mathsf{node}_g(\vec{d}; t, e). \tag{record}$$

The reduction relation is the compatible closure of these axioms under every type and term former. In particular, it reduces under binders, in dependent indices, in each stamp field, in every gate parameter, and in every retained node field. The full one-hole-context grammar is in appendix B.

The record rule deliberately drops the origin proof but retains both the payload and event. A private node has no head reduction. Hence a consumed stamp cannot later erase its event field while the corresponding node survives. The retained event may itself reduce or be transformed by a later substitution.

Definitional equality $t \equiv_\rho u : A$ and type equality $A \equiv_\rho B$ are the least typed, reflexive, symmetric, transitive, and congruent relations containing $\longrightarrow$. There are no eta-rules and no user equations in the core. The mechanization reads $\equiv$ as conversion, the equivalence closure of $\longrightarrow$ on raw expressions, and asks in the conversion rule that the target type be formed. By confluence and subject reduction a conversion between formed types passes through formed types only, so on formed types the two readings agree.

7.2 Normal forms and event support

A term is normal when it contains no instance of equations (β_q) to (record). For a normal term n, define its event support structurally by

$$\mathsf{events}(\mathsf{node}_g(\vec{d}; t, e)) = \{e\} \cup \mathsf{events}(\vec{d}) \cup \mathsf{events}(t) \cup \mathsf{events}(e),$$
$$\mathsf{events}(k(\vec{t})) = \bigcup_i \mathsf{events}(t_i) \quad \text{for every other constructor } k.$$

The event field is searched too: a node may stand inside the event of another, and its event is support as well. Events inside an unused argument erased by beta reduction do not occur in the normal support. Events inserted by a contracted gate do occur, because a node has no event-erasing rule.

Definition 7.1 (Residual record trace)**.** Instrument each instance of equation (record) with a fresh metatheoretic label ℓ on the node it creates. Labels are ignored by typing and reduction, are copied when substitution duplicates a node, and disappear only when the whole labelled occurrence is discarded. For $\pi : t \longrightarrow^* n$, define $\mathsf{Cross}^{\downarrow}(\pi, n)$ to be the multiset of triples (ℓ, e, e') such that the contraction labelled ℓ stored event e, and a residual of that node occurs in n with final event field e'.

The labels are only a residual-accounting device; erasing them gives exactly the reduction relation above. The multiset records repeated contractions and duplication, whereas $\mathsf{events}(n)$ records set-valued final support. Because n is normal, every recorded residual event e' is normal. For a top-level closed record not moved through a binder, $e' = \mathsf{nf}(e)$; the more general residual formulation also covers contraction under binders followed by substitution.

It would be incorrect to compare the raw event at contraction with final support by syntactic equality. For example, an event may be a beta-redex that is retained by the record step and

normalized only afterward. The residual trace records precisely the final representative that support can contain.

8 Structural metatheory

8.1 Effects as path semantics

Every effect-bearing rule edge is labelled by the coefficient displayed in its effect equation. Ordinary computational premises have edge 0; the domain edge of Ty-Pi and argument edge of App have edge q; and the stamp premise of a gate and the payload and event premises of a node have edge 1. Formation premises used only to validate a type annotation are separate accepted judgments and contribute no term-effect path. Equivalently, their edge in the term derivation is ∞. An ∞-edge kills a path. For an atom occurrence path p, let $\mathsf{cost}(p)$ be the $\otimes$-product, hence the maximum, of its edge labels. A path is *live* when its cost is below ∞.

Lemma 8.1 (Path characterization)**.** *If a derivation has inferred effect ε, then for every atom z,*

$$\varepsilon(z) = \bigoplus_{p : z \rightsquigarrow \mathrm{root}} \mathsf{cost}(p),$$

where the empty sum is ∞.

Proof. Induction on the last rule. A variable or constant has the single empty path of cost 0. An ordinary constructor takes the minimum over the paths in its premises. Application prefixes every argument path by q, producing $q \otimes \eta$. Abstraction deletes paths beginning at its bound variable. The gate and node cases prefix exactly the privileged paths by 1. These are precisely the effect equations in sections 6.5 and 6.6. □

Theorem 8.2 (Principal and decidable effects)**.** *For a fixed well-formed signature, context, annotated expression, and result role, its effect is unique when it is typable. Effect inference and policy validation are decidable. For an unannotated lambda, elaboration uniquely chooses the maximal admissible grade $q = \theta(x)$, where θ is the principal body effect. An explicit annotation is accepted exactly when $D(\sigma, \rho) \le q \le \theta(x)$.*

Proof. The syntax determines the last rule. Each effect equation uses finite pointwise minimum, maximum, restriction, and unit-vector construction. Conv does not change the effect, so the candidate principal effect is computed independently of the later conversion decision. The lambda equation tests $q \le \theta(x)$; maximal source elaboration chooses equality. The binder-policy test and ordinary policy validation are finite comparisons. □

8.2 Substitution

For ε over $\Gamma, x :_\sigma A$ and u with effect η, define

$$\varepsilon\langle x := \eta \rangle = (\varepsilon \setminus x) \oplus (\varepsilon(x) \otimes \eta). \tag{8.1}$$

For a simultaneous substitution $\theta : \Delta \to \Gamma$, let $M_\theta(y, z)$ be the effect at source atom z of the term replacing y. Matrix action is over the min–max semiring:

$$(M_\theta \star \varepsilon)(z) = \bigoplus_{y \in \Gamma} \varepsilon(y) \otimes M_\theta(y, z), \tag{8.2}$$

with the signature-atom components of ε included unchanged.

Lemma 8.3 (Exact single substitution)**.** *Suppose*

$$\mathfrak{S};\Gamma, x :_\sigma A \vdash_\rho J \triangleright \varepsilon \quad \textit{and} \quad \mathfrak{S};\Gamma \vdash_\sigma u : A \triangleright \eta,$$

where J is either a type or a term judgment. Then

$$\mathfrak{S};\Gamma \vdash_\rho J[u/x] \triangleright \varepsilon\langle x := \eta\rangle.$$

Proof. Simultaneous induction on type formation and term typing. The variable case is the definition of $\mathbf{e}_x$. Ordinary constructors use distributivity of $\otimes$ over $\oplus$. In the application case the induction hypotheses and associativity of $\otimes$ give

$$q \otimes (\varepsilon(x) \otimes \eta) = (q \otimes \varepsilon(x)) \otimes \eta.$$

Binder cases use alpha-renaming and effect restriction. Gate and node cases are the application calculation with fixed prefix 1. In the lambda case the bound name is chosen fresh for the substituend, so substitution does not change its body-effect component and preserves $q \leq \theta(x)$. Substitution in dependent result types uses the simultaneous type induction. □

Theorem 8.4 (Policy-preserving substitution)**.** *If both premises of Lemma 8.3 are policy valid and D is path closed, then the substituted conclusion is policy valid. Simultaneous substitution has effect $M_\theta \star \varepsilon$, and composition of substitutions is associative.*

Proof. Consider a source role τ, the substituted variable role σ, and result role ρ. Validity gives effects at least $D(\tau, \sigma)$ and $D(\sigma, \rho)$. Their composite is at least

$$D(\tau, \sigma) \otimes D(\sigma, \rho) \geq D(\tau, \rho)$$

by path closure. Minimum over multiple substituted paths preserves the same lower bound. Associativity is min–max matrix associativity. □

Weakening is the special case that extends an effect by $\boldsymbol{\infty}$. Exchange is admissible for independent declarations, and strengthening is admissible when the deleted component is $\boldsymbol{\infty}$.

8.3 Preservation

Effects are ordered pointwise. Moving upward means that a dependency becomes more mediated or disappears.

Theorem 8.5 (Subject reduction with effect improvement)**.** *If $\mathfrak{S};\Gamma \vdash_\rho t : A \triangleright \varepsilon$ and $t \longrightarrow t'$, then*

$$\mathfrak{S};\Gamma \vdash_\rho t' : A \triangleright \varepsilon' \qquad \textit{for some } \varepsilon \leq \varepsilon'.$$

Consequently policy validity is preserved. The analogous result holds for reduction in types.

Proof. Compatible cases use monotonicity of $\oplus$ and $\otimes$. For beta, let θ be the body effect. The redex effect contains $q \otimes \eta$ with $q \leq \theta(x)$, while Lemma 8.3 gives the contractum effect $(\theta \setminus x) \oplus (\theta(x) \otimes \eta)$. The two beta effects are equal for maximally elaborated lambdas and otherwise the contractum effect is greater. Projection may discard one component and therefore only raises its effect. In the record step the origin-proof component disappears; all other components retain exactly the same prefixes.

The only subtle compatible case is reduction beneath a lambda. If its body effect improves from θ to θ', then $q \leq \theta(x) \leq \theta'(x)$, so the same annotation remains admissible; the external effect improves from $\theta \setminus x$ to $\theta' \setminus x$. This is the reason Lam uses a stable lower bound rather than equality. Type preservation follows from substitution in the beta and projection cases and from the gate declaration's fixed result type in the record case. □

8.4 Normalization, confluence, and canonical forms

The proofs in this subsection concern exactly the syntax in section 6. They do not cover arbitrary module rewrite rules. We give the reducibility argument because the record rule is not an instance of the ordinary ΠΣ metatheory.

Simple skeletons. Erase indices, roles, and grades from a well-formed type but retain its outer computational shape:

$$\tau ::= \iota_a \mid \iota_E \mid \iota_O \mid S\tau \mid \tau \Rightarrow \tau \mid \tau \times \tau.$$

Write $\lfloor A \rfloor$ for this skeleton. Family applications map to ι_a, Event to ι_E, every origin type to ι_O, stamps to $S\lfloor A \rfloor$, dependent products to arrows, and dependent sums to products.

Lemma 8.6 (Skeleton invariance)**.** *If $A \longrightarrow A'$, $A \equiv A'$, or $A[u/x]$ is defined, then, respectively,*

$$\lfloor A \rfloor = \lfloor A' \rfloor, \qquad \lfloor A \rfloor = \lfloor A' \rfloor, \qquad \lfloor A[u/x] \rfloor = \lfloor A \rfloor.$$

Proof. Reduction occurs only in recursive type positions and term indices; no rule changes a type-former head. The first and third statements follow by structural induction. The equality statement follows by induction over the generation of definitional equality. □

Let SN be the set of raw terms having no infinite reduction. Define reducibility sets $\mathcal{R}_\tau$ by induction on τ:

$$\begin{aligned}
\mathcal{R}_\iota &= \mathsf{SN}, \\
\mathcal{R}_{S\tau} &= \{t \in \mathsf{SN} \mid t \longrightarrow^* \mathsf{stamp}(u, e, o) \Rightarrow u \in \mathcal{R}_\tau \text{ and } e, o \in \mathsf{SN}\}, \\
\mathcal{R}_{\tau \Rightarrow \upsilon} &= \{t \in \mathsf{SN} \mid u \in \mathcal{R}_\tau \Rightarrow t\,u \in \mathcal{R}_\upsilon\}, \\
\mathcal{R}_{\tau \times \upsilon} &= \{t \in \mathsf{SN} \mid \mathsf{fst}(t) \in \mathcal{R}_\tau \text{ and } \mathsf{snd}(t) \in \mathcal{R}_\upsilon\}.
\end{aligned}$$

A term is *neutral* when it is not a lambda, pair, or stamp and has no head redex. Thus variables, public constants, private nodes, their neutral spines, and a gate with a neutral credential are neutral; a gate whose credential is visibly a stamp is not.

Lemma 8.7 (Reducibility closure)**.** *For every skeleton τ:*

(i) $\mathcal{R}_\tau \subseteq \mathsf{SN}$*;*

(ii) if $t \in \mathcal{R}_\tau$ and $t \longrightarrow t'$, then $t' \in \mathcal{R}_\tau$;

(iii) if r is neutral and every one-step reduct of r is in $\mathcal{R}_\tau$, then $r \in \mathcal{R}_\tau$;

(iv) beta, projection, and record head expansion preserve reducibility when their contracta and immediate computational components are reducible, and any discarded proof component is strongly normalizing; and

(v) stamps with reducible payload and strongly normalizing event and origin fields are in $\mathcal{R}_{S\tau}$.

Proof. Simultaneous induction on τ. Clause (i) is part of every defining clause. For (ii), an infinite reduction from t' would extend to one from t. At an arrow, fix $u \in \mathcal{R}_\sigma$; the step $t\,u \longrightarrow t'\,u$ and the induction hypothesis at the codomain give the arrow clause. At a product, use the corresponding steps under both projections. At a stamp skeleton, prepend $t \longrightarrow t'$ to any reduction of t' to a stamp.

For (iii), all immediate reducts are strongly normalizing, so the neutral term is strongly normalizing. At an arrow, fix $u \in \mathcal{R}_\sigma$. The term $r\,u$ is neutral. Its reducts are $r'\,u$, handled by the premise and the arrow clause, or $r\,u'$, handled by subsidiary induction on the finite reduction

height of u. The product case applies the same argument to $\mathsf{fst}(r)$ and $\mathsf{snd}(r)$. For $S\sigma$, any reduction from r to a stamp begins with a reduct covered by the premise. This proves neutral closure.

For (iv), induct on the result skeleton and, secondarily, on the sum of the reduction heights of the immediate components. A one-step reduct either is the designated contractum or reduces a component and decreases that height; clauses (ii) and (iii) close the cases. For (v), every reduction of a stamp is in one of its fields. Their finite reduction heights give strong normalization, and clause (ii) preserves the required component membership at every canonical stamp reduct. □

Lemma 8.8 (Gate and node reducibility)**.** *Suppose all ordinary parameters of a registered gate are reducible at their skeletons.*

(i) If t, e are reducible at the payload and event skeletons, then $\mathsf{node}_g(\vec{d}; t, e) \in \mathcal{R}_{\lfloor B \rfloor}$.

(ii) If $s \in \mathcal{R}_{S\lfloor A \rfloor}$, then $\mathsf{gate}_g(\vec{d}; s) \in \mathcal{R}_{\lfloor B \rfloor}$.

Proof. For (i), use neutral closure and induction on the sum of the finite reduction heights of $\vec{d}, t, e$. A node has no head step, and every reduct lowers that sum.

For (ii), induct on the reduction heights of $\vec{d}, s$. Reducing a parameter or the credential lowers the measure. If the credential is neutral, neutral closure applies. If $s = \mathsf{stamp}(t, e, o)$, the definition of $\mathcal{R}_{S\lfloor A \rfloor}$ gives $t \in \mathcal{R}_{\lfloor A \rfloor}$ and $e \in \mathsf{SN}$; the record contractum is reducible by (i), and record head expansion from Lemma 8.7(iv) gives the redex. The origin field is strongly normalizing and is discarded, so it creates no additional case. □

A substitution γ is *reducible for* Γ when, in context order, it maps each $x :_\rho A$ to a term in $\mathcal{R}_{\lfloor A\gamma \rfloor}$. Fresh variables give the identity reducible substitution by neutral closure.

Lemma 8.9 (Fundamental lemma)**.** *Let γ be reducible for Γ.*

(i) If $\mathfrak{S}; \Gamma \vdash_\rho A$ type, then $A\gamma$ is strongly normalizing.

(ii) If $\mathfrak{S}; \Gamma \vdash_\rho t : A$, then $t\gamma \in \mathcal{R}_{\lfloor A \rfloor}$.

Proof. Simultaneous induction on the accepted formation and typing derivations. For formation, each type former has finitely many recursive type fields and typed term indices. The induction hypotheses make those fields strongly normalizing; the multiset of their reduction heights strictly decreases at every type step. Under a binder, extend γ by the fresh neutral variable. This proves (i).

For (ii), variables are covered by reducibility of γ, and constants by neutral closure. For a lambda, extend γ by an arbitrary reducible argument and use the term induction hypothesis on the body; beta head expansion in Lemma 8.7 proves the arrow clause. The grade inequality and role annotations are computationally inert. For an application use the arrow clause. Pairing and projections use the product clause and projection expansion. A stamp uses Lemma 8.7(v). Gate and node terms use Lemma 8.8. Conversion is harmless by Lemma 8.6. These are all term rules. □

Theorem 8.10 (Strong normalization)**.** *Every well-typed* RATT_c *type and term is strongly normalizing.*

Proof. Use the identity reducible substitution in Lemma 8.9. Term reducibility implies strong normalization by Lemma 8.7(i), and the formation clause gives the type case. □

Local peaks. We spell out the interaction between beta reduction and records rather than appealing only to orthogonality.

Lemma 8.11 (Residuals of one step)**.** *If $r \longrightarrow r'$, then $r[u/x] \longrightarrow^* r'[u/x]$. If $u \longrightarrow u'$, then $r[u/x] \longrightarrow^* r[u'/x]$, reducing once in every residual of u. The same statements hold for substitution into types.*

Proof. Structural induction on r. The variable case is immediate. Binder cases use alpha-renaming; every other case uses compatible closure. The second statement may require zero steps when x is absent and finitely many steps when substitution duplicates u. □

Lemma 8.12 (Local confluence)**.** *If a well-typed expression r reduces in one step to both r_1 and r_2, then there is v with $r_1 \longrightarrow^* v \longleftarrow^* r_2$.*

Proof. Compare the two redex positions. Disjoint positions commute. Equal positions use the same unique head rule. It remains to consider a redex strictly inside another head redex.

For beta, a body step is transported through substitution by the first part of Lemma 8.11; an argument step is transported to all of its residuals by the second part. A step in the lambda annotation is discarded by both beta paths. For a projection, a step in the selected component is retained and commutes, while a step in the discarded component is erased on both paths. For a record, steps in gate parameters, payload, or event are retained in the corresponding node fields and commute. A step in the origin proof is discarded by both paths. No head rule overlaps a different head rule: lambda, pair, and gate-with-stamp have distinct heads. Compatible reductions in type indices and annotations are the same retained or discarded context cases. This exhausts the grammar. □

Theorem 8.13 (Confluence)**.** *Typed reduction in $\mathsf{RATT}_{\mathrm{c}}$ is confluent.*

Proof. By Theorem 8.10 every typed reduction relation is terminating, and by Lemma 8.12 it is locally confluent. Newman's lemma therefore gives confluence. □

Canonical forms. For normal annotations and fields, define neutral and normal terms mutually:

$$
\begin{aligned}
h &::= x \mid c \mid \mathsf{node}_g(\vec{n}; n, e) \mid \mathsf{gate}_g(\vec{n}; r),\\
r &::= h \mid r\, n \mid \mathsf{fst}(r) \mid \mathsf{snd}(r),\\
n &::= r \mid \lambda^q(x :_\sigma A).n \mid (n, n) \mid \mathsf{stamp}(n, e, o).
\end{aligned}
$$

In the gate head, the credential r is neutral, so the record redex is absent. Event and origin fields in the grammar are themselves normal terms.

Theorem 8.14 (Canonical forms)**.** *Every well-typed normal term has exactly one of the preceding normal forms. Moreover, for closed normal terms:*

(i) a term of $\mathsf{Stamped}_\sigma(A)$ is a stamp;

(ii) a term of Event is an event constant from a unique certificate or attestation entry, and a term of $\mathsf{Origin}_A(e, t)$ is that entry's matching origin constant;

(iii) a term of $\Pi^q(x :_\sigma A).B$ with $q \in \{1, \omega\}$ is a lambda; and

(iv) a closed neutral at a protected grade-free type is headed by an ordinary public constant or a private node; it is never headed by a public gate.

Proof. The grammar statement is structural induction on the term. A normal application cannot have a lambda head, a normal projection cannot have a pair head, and a normal gate cannot have a stamp credential; these are exactly the four omitted redex shapes.

For (i), a closed neutral cannot be variable-headed. Primitive and certificate-payload declarations have ordinary ultimate result heads, attestations introduce no payload constant, gates have protected results, and nodes have their gates' protected results. Hence none has stamped type. The only remaining normal introduction is a stamp. For (ii), the signature rules prohibit primitive, gate, or payload constants with ultimate head Event or Origin; certificate and attestation entries are their only sources. Fresh qualified names give unique ownership. Type-head injectivity, which follows from Lemma 8.6 and confluence, forces an origin constant's event and payload to match the expected normal fields.

For (iii), a closed neutral head of positive or absent function grade cannot be a primitive or certificate payload because their types are grade-free; an attestation adds only event and origin constants, and a gate or node has a grade-free result. Thus the normal form is a lambda. For (iv), a closed gate credential is a stamp by (i), making the gate a redex. The remaining closed neutral heads are precisely the two listed classes; grade-freedom excludes a nonzero latent application edge in their spine. □

Corollary 8.15 (Canonical credentials)**.** *Every closed normal term of a stamped type is headed by* stamp*. Its event and origin come from one unique evidence entry. For a certificate its payload is the fresh payload constant; for an exact attestation its payload is definitionally equal to the old closed term named by that entry.*

Proof. Apply Theorem 8.14(i–ii) and type-head injectivity for $\mathsf{Origin}_A(e, t)$. Its normalized indices distinguish the certificate and exact-attestation cases. □

Theorem 8.16 (Decidable checking and conversion)**.** *Over a finite well-formed signature:*

(i) definitional equality of two well-formed types, or of two well-typed terms, is decidable;

(ii) the effect of a typable expression is computed with its type, and policy validation reduces to the binder tests $D(\sigma, \rho) \leq q$ and $q \leq \theta(x)$ and to the test that an origin's payload type is not a rendering type;

(iii) signature formation, context formation, type formation and type checking are decidable on the bidirectional fragment of appendix C, which contains every well-typed normal form.

Proof. (i) By Theorems 8.10 and 8.13, two well-typed expressions are definitionally equal exactly when their normal forms are equal. Contracting the leftmost redex until none is left computes the normal form, and the iteration terminates by strong normalization. (ii) Effects are unique by Theorem 8.2. Over a well-formed signature the validity side condition of every rule follows from the validity of its premises and from the binder tests, except for Ty-Origin over a rendering type, which is never valid: a rendering type has grade 0 at a rendering family, of demand $\boldsymbol{\infty}$ into W. (iii) The procedure of appendix C answers with derivations and recurses on the size of the expression; it accepts exactly the derivable judgments of the fragment (Proposition C.2). □

9 Gate adequacy

9.1 Static path mediation

Theorem 9.1 (Path mediation)**.** *Let a policy-valid type or term derivation have result role ρ and principal effect ε. For every source atom z of role σ:*

(i) if $D(\sigma, \rho) = 1$, every live effect path from z to the root contains a registered gate or node edge, a grade-1 Ty-Pi domain edge, or a latent grade-1 App edge; and

(ii) if $D(\sigma, \rho) = \infty$, there is no live effect path from z to the root: every such derivation path contains an ∞-edge.

These path statements are stable under policy-valid substitution. Reduction may discard paths but cannot create a path below the policy demand.

Proof. By Lemma 8.1, $\varepsilon(z)$ is the minimum path cost. Validity with demand 1 implies that every live cost is at least 1. Inspection of the effect equations shows that only a gate edge, node edge, grade-1 Ty-Pi edge, or grade-1 App edge can raise a live path from 0 to 1. Demand ∞ implies that every path has cost ∞, hence contains an ∞-edge. Stability is Theorems 8.4 and 8.5. □

The second clause is an effect statement, not syntactic erasure. For example, if $\vdash_{\mathsf{R}} N\ \mathsf{type}$ and $\vdash_{\mathsf{F}} B\ \mathsf{type}$, then the normal type $\Pi^{\infty}(x :_{\mathsf{R}} N).B$ still contains N, although its occurrence has no live path. Likewise, a bare Π^{1}-type supplies a type-formation edge but has no term body to summarize. Operational realization therefore requires a term-level canonical-form statement.

Lemma 9.2 (Positive-function realization)**.** *Every closed normal term of type $\Pi^{1}(x :_{\sigma} A).B$ at result role ρ is a lambda $\lambda^{1}(x :_{\sigma} A).t$, and the principal effect of x in t is either 1 or ∞. If x occurs on a live body path, that effect is 1. Consequently, applying such a closed function exposes either the gated body path summarized by its grade or an argument that the body discards.*

Proof. A closed normal function cannot be headed by a variable. Primitive and certificate declarations are grade-free and have ordinary ultimate heads; gate and node results are grade-free. Hence none can have the displayed positive function type. The only remaining canonical form is a lambda, and Lam requires $1 \leq \theta(x)$. The only possibilities are 1 and ∞, and liveness excludes the latter. □

The word "closed" in Lemma 9.2 does not eliminate neutral applications beneath an unapplied outer lambda. For example, a bound variable may itself be assumed to have positive function type. The dynamic theorem below therefore concerns grade-free result types, where such latent positive interfaces cannot remain in the observed normal form.

Call a public declaration occurrence in an expression *effect relevant* when its path in the principal derivation is live in the sense of section 8, that is, when its path cost is below ∞. Bound variables are handled by the binder-policy premise but are not public extension atoms. Occurrences confined to type annotations behind ∞-edges are syntactically present but are not effect relevant.

Lemma 9.3 (No latent protected escape)**.** *Let $\vdash_{\rho} n : A$ be closed and normal, where $\rho \in \{\mathsf{F}, \mathsf{P}\}$ and $\mathsf{gradefree}(A)$. Outside private nodes:*

(i) no effect-relevant public declaration path can end in a grade- 1 *or grade-∞ neutral application edge; and*

(ii) a normal public gate cannot carry an effect-relevant closed public declaration occurrence in its credential.

Proof. Use the normal/neutral grammar of Theorem 8.14, inducting simultaneously on n and the grade-free type through which it is checked. For (i), an application with nonzero latent edge cannot have a lambda head, since it would be a beta-redex. Primitive and certificate types are grade-free, and gate and node result types are grade-free. Its neutral head must therefore be a bound variable whose declared type contains that nonzero grade. The type is a subexpression of an enclosing binder domain, contradicting the recursive definition of $\mathsf{gradefree}(A)$.

For (ii), a normal gate has a neutral, rather than stamped, credential. No public declaration may have ultimate stamped result, and no gate or node has witness result. Its neutral head is

therefore a bound variable whose ultimate result role is W. Along the enclosing protected lambda, Ty-Pi requires $D(\mathsf{W}, \rho) \leq q$, hence $1 \leq q$, contradicting grade-freedom. The same argument applies when the public occurrence is in an ordinary argument of that neutral credential head. The induction passes through ordinary grade-zero lambdas, pairs, and neutral spines; those constructors introduce no nonzero edge. □

Lemma 9.4 (Grade-free normal-form mediation)**.** *Let $\vdash_\rho n : A$ be closed and normal, with $\rho \in \{\mathsf{F}, \mathsf{P}\}$ and $\mathsf{gradefree}(A)$. Every effect-relevant occurrence in n of a public atom z of role σ satisfying $D(\sigma, \rho) = 1$ lies below a private node edge. No occurrence with policy demand $\boldsymbol{\infty}$ is effect relevant in n.*

Proof. By Theorem 9.1, a live demand-1 occurrence path contains a node edge, a public-gate edge, or a nonzero latent application edge; formation premises for type annotations are not term-effect paths. The latter two possibilities outside nodes are excluded by Lemma 9.3. Induction through the canonical grammar then places the occurrence below the first private node on its path. For demand $\boldsymbol{\infty}$, path mediation says every derivation path has an $\boldsymbol{\infty}$-edge and hence is not live. □

The restrictions on primitive grades and gate result types are essential. If a module could simply declare $c :_{\mathsf{F}} \Pi^1(x :_{\mathsf{W}} A).B$, the type would assert mediation without any term that emits a record. Likewise, if a gate node could have a result containing an opaque positive-grade function, its existing record could be misused to justify unrelated future arguments. $\mathsf{RATT}_{\mathrm{c}}$ therefore admits a closed canonical function of positive grade only as a lambda. Maximal source elaboration gives that lambda its exact principal parameter grade; after reduction, a stable positive annotation may conservatively remain on a body that has become unused, but every live parameter path still has grade 1. Open terms may assume positive function variables, which is why the dynamic result has a grade-free observation boundary. A registered gate remains a saturated term former that such a body may invoke.

9.2 Dynamic records

Lemma 9.5 (Record persistence)**.** *If $(\ell, e, e') \in \mathsf{Cross}^{\downarrow}(\pi, n)$, then a residual node labelled ℓ occurs in n, its final event field is e', and $e' \in \mathsf{events}(n)$. The field e' is obtained from e by the reductions and substitutions along that residual; if e is closed at contraction, then $e' = \mathsf{nf}(e)$. Before normalization, every reduct after the labelled contraction contains at least one residual of that node unless a surrounding redex discards every residual of the protected subterm.*

Proof. The record rule inserts e in the labelled node. No rule removes an event field from a surviving node. Compatible reduction may reduce or substitute inside that field, move the node, or duplicate it through substitution. It can remove all residuals only by discarding the containing subterm. The definition of $\mathsf{Cross}^{\downarrow}$ records the final event field e' of each surviving residual, and the structural definition of events includes that field. Residual propagation applies exactly the ambient reductions and substitutions to the stored field. When e is closed, substitution has no effect; normality of n and confluence give $e' = \mathsf{nf}(e)$. □

Lemma 9.6 (Public node ancestry)**.** *If t is public and $\pi : t \longrightarrow^* n$, every private node occurrence in n is a residual of at least one record contraction in π.*

Proof. Induct over π. The initial expression has no node. Ordinary beta and compatible steps can move, duplicate, reduce inside, or discard existing node residuals but cannot create a new node head. Projection can only retain or discard them. The record rule is the unique rule whose contractum introduces a node, and its fresh label identifies every later copy. Hence every final node has the asserted ancestor. □

Theorem 9.7 (Static–dynamic gate adequacy)**.** *Let $\vdash_\rho t : A$ be a closed public, policy-valid term, with $\rho \in \{\mathsf{F}, \mathsf{P}\}$ and $\mathsf{gradefree}(A)$, and let $\pi : t \longrightarrow^* n$ be a normalization sequence. Then:*

(i) every effect-relevant occurrence in n whose policy demand is 1 lies beneath a residual node created by a registered gate contraction in π; and

(ii) if $(\ell, e, e') \in \mathsf{Cross}^{\downarrow}(\pi, n)$, then $e' \in \mathsf{events}(n)$.

In particular, a successful exceptional crossing cannot be both computationally relevant to the result and absent from its normal provenance.

Proof. By Lemma 9.4, every relevant positive occurrence in n lies below a private node. Since t is public, every such node has at least one record-step ancestor in π by Lemma 9.6; residual labels make this ancestry explicit even when substitution duplicates the node. This proves (i). Part (ii) is Lemma 9.5. Confluence makes n, and hence its structural event support, independent of the chosen normalization sequence, up to alpha-equivalence. □

The qualification "contributing to the result" is necessary. A gate may be constructed in an argument that an enclosing function ignores. Normalization correctly removes both that computation and its record.

10 Modules and extension theorems

10.1 Module forms

A module over $\mathfrak{S}$ is a finite well-formed signature suffix $\mathcal{M}$ whose public names share an owner and are fresh for $\mathfrak{S}$. Concatenation is written $\mathfrak{S} + \mathcal{M}$. The core distinguishes the following syntactic classes; membership is decidable by inspection of the suffix.

Rendering-only Every new family and primitive has result role R. There is no certificate bundle, exact attestation, or gate.

Fresh-witness-family Every new family has role W, and every new primitive is a constructor whose result is headed by one of those new families. There is no certificate bundle, exact attestation, or gate.

Evidence-inhabitant The nonempty suffix consists of any mixture of certificate bundles for old closed types at role W and exact attestations of old closed public terms at old closed witness-data types. It adds no family, ordinary primitive, or gate. An attested payload term is old.

Gate module The suffix contains one or more registered gates and may therefore add new protected construction power. Its guarantees are mediation, record adequacy, normalization, and decidable checking, not conservativity.

A script generator is an ordinary primitive at role S. Script equations are not a declaration form in RATT_c; they belong to a separate typed setoid or categorical presentation. This prevents an extension from collapsing old script equality through fresh intermediaries.

10.2 Old terms and equality reflection

An expression is *old* when all of its public declaration atoms belong to $\mathfrak{S}$. An extension is term-conservative at an old closed type A of role ρ when every closed public extended term of that type is definitionally equal to an old term. It reflects equality when equality of two old public terms in the extension implies their equality in the base. Public reachability is essential: the internal NODE rule is a reduction target, not a source-language constructor.

The absence of internal universes is used exactly here. A new family symbol cannot be packaged as data in an old type, and an old gate payload cannot be instantiated by a family declared only later.

Lemma 10.1 (Old-head and evidence-entry inversion)**.** *Let $\mathfrak{S} + \mathcal{M}$ add no protected family, protected primitive, or gate.*

(i) A new rendering primitive has the rendering role, and a new fresh-family constructor has ultimate head its new witness family. Neither heads a closed normal term at an old grade-free type of a non-rendering role.

(ii) If $\mathcal{M}$ adds neither a certificate bundle nor an exact attestation, every closed normal stamp at an old stamped type is old.

(iii) If $\mathcal{E}$ is evidence-inhabitant and a closed normal stamp at an old stamped type contains a new atom, then for one unique evidence entry its normal fields have one of the forms

$$\mathsf{stamp}(a_d, e_d, o_d) \quad \textit{or} \quad \mathsf{stamp}(\mathsf{nf}(t_q), e_q, o_q),$$

for a certificate or exact attestation, respectively.

(iv) If a public closed term reduces to a node residual containing an $\mathcal{E}$-atom in a retained payload or event, its normal event field is owned by $\mathcal{E}$. A new retained payload atom occurs only in the certificate case and is paired with that certificate's event.

Proof. For (i), a rendering primitive has the rendering role, and the role of a term is the role of its head. A rendering primitive may construct an old rendering family, so its ultimate head need not be new; only its role keeps it out. Signature order prevents an old declaration type from mentioning a new family. Type-head injectivity follows because reduction never changes a type-former head and confluence gives unique normal types. Hence a neutral spine headed by a fresh-family constructor cannot synthesize an old result head.

For (ii), Theorem 8.14 makes the credential a stamp and its closed normal origin field an evidence-entry origin constant. Because the extension adds no certificate or attestation entry, that origin is old. Injectivity of its normal Origin type forces the event and payload fields to be old as well.

For (iii), the same canonical-form argument applies. The evidence extension has no new constants other than certificate triples and attestation event–origin pairs. If the origin constant is o_d, its declared type is $\mathsf{Origin}_A(e_d, a_d)$, so injectivity forces the stamp's event and payload to e_d and a_d. If it is o_q, its type is $\mathsf{Origin}_A(e_q, t_q)$, so the normal event and payload are e_q and $\mathsf{nf}(t_q)$. The module premise makes t_q old, and qualified freshness makes the entry unique.

For (iv), use Lemma 9.6 to trace the residual to its record step. Compose the substitutions encountered by that residual between the contraction and the closed final term, and apply the composite substitution to the creating stamp, including its then-discarded origin field. This produces a closed well-typed stamp whose retained payload and event normalize to the final residual fields. Confluence and (iii) align a new payload atom with its certificate event. In the attestation case the payload is old and the new retained atom is the event itself. Substitution may duplicate the residual and reduction may normalize its fields, but neither changes the qualified event constant; Lemma 9.5 carries it to the final node. □

Lemma 10.2 (Grade-free extension support)**.** *Let A at role ρ be old, closed, and grade-free, let $\vdash_\rho t : A$ be closed and public in an extension that adds no protected family, protected primitive, or gate, and put $n = \mathsf{nf}(t)$.*

(i) If the extension is rendering-only and $\rho \neq \mathsf{R}$, then n is old.

(ii) If it is fresh-witness-family and $\rho \in \{\mathsf{F}, \mathsf{P}\}$, then n is old.

(iii) If it is evidence-inhabitant and $\rho \in \{\mathsf{F}, \mathsf{P}\}$, every new atom in n occurs in a residual of an old gate node, and that node's event field is a qualified event constant owned by the extension.

Proof. Induct on the canonical grammar of n, simultaneously on the old grade-free type against which each subterm is checked. By Lemma 10.1(i), a new rendering or fresh-witness declaration cannot be the head at an old type. A new atom could therefore survive only as an argument of an old neutral head, in a stamp, or in a private node. Grade-free latent-edge exclusion (Lemma 9.3) removes the first possibility whenever the policy requires a nonzero edge. Ordinary grade-zero lambdas and pairs pass the induction hypothesis to their old component types.

For (i), a rendering occurrence has demand ∞ into every non-rendering result. A closed old gate credential is old by Lemma 10.1(ii), and Lemma 9.4 says it has no live path. The only syntactic place that could retain it would be an ∞-graded neutral application, which Lemma 9.3 excludes. Thus n is old. At the witness and script roles, where those two lemmas are not stated, the induction of the first paragraph applies directly: every subterm of n is checked against an old grade-free type of a non-rendering role, in a context of such variables, and the grade-free gate interface of Sig-Gate (remark 10.4) carries the induction through gate parameters, payloads and node fields.

For (ii), a fresh witness occurrence has demand 1 into a protected result, so Lemma 9.4 places it below an old node. Trace that node to its creating old gate using Lemma 9.6. The creating credential is at an old stamped type and is old by Lemma 10.1(ii). Node parameters are handled by the outer induction, so no fresh occurrence can enter a retained field. Therefore n is old.

For (iii), mediation places every new payload, event, or origin occurrence beneath an old node. Origin fields are discarded by the record step, and Lemma 10.1(iv) says that the nearest residual retaining a new payload has the matching extension-owned event. New event occurrences are already such retained event fields. Hence every new atom has a witnessing old node with an $\mathcal{E}$-owned event field. □

Remark 10.3 (Why grade-freedom is necessary)*.* The restriction is not cosmetic. Let $\vdash_{\mathsf{R}} G\ \mathsf{type}$, $\vdash_{\mathsf{W}} W\ \mathsf{type}$, and $\vdash_{\mathsf{F}} B\ \mathsf{type}$ be old. Put

$$F_R = \Pi^{\infty}(x :_{\mathsf{R}} G).B, \qquad F_W = \Pi^{1}(x :_{\mathsf{W}} W).B.$$

A rendering-only extension adding $r : G$ creates the closed normal, non-old term $\lambda^0(f :_{\mathsf{F}} F_R).f\, r$ of the old non-rendering type $\Pi^0(f :_{\mathsf{F}} F_R).B$. Likewise, an evidence bundle adding $a_d : W$ creates $\lambda^0(f :_{\mathsf{F}} F_W).f\, a_d$ at an old form-role type without yet producing a record. Both dependencies are latent behind a neutral higher-order call, and both result types contain a nonzero grade. Thus unrestricted higher-order conservativity and traceability would be false. Lean checks both examples over a signature of the three families: no old term is convertible to the first, and no event of the bundle occurs in the second (table 8).

Remark 10.4 (Grade-free gate interfaces)*.* The same latent dependency can enter through a gate parameter. If a gate could take a parameter of type $P = \Pi^0(h :_{\mathsf{F}} F_R).X$, with F_R as above and X an old form type, then after the rendering-only extension adding $r : G$ the public term $\mathsf{gate}_g(\lambda^0(h :_{\mathsf{F}} F_R).h\, r;\, c)$ of an old grade-free type contracts to a node whose parameter retains

r, and Lemma 10.2(i) fails. With F_W in place of F_R, an evidence bundle's payload a_d enters a node whose event is old, and Theorem 10.7 fails. Sig-Gate therefore requires the parameter types of a gate to be grade-free, and it requires the payload type to be grade-free as well, so that the extension arguments treat payloads as they treat parameters. An earlier version required only the result type to be grade-free; every gate of this paper meets the stronger condition. Under the earlier rule both failures occur; Lean checks them over a signature with two such gates, which that rule admits and Sig-Gate rejects (table 8).

Theorem 10.5 (Rendering noninterference)**.** *Let $\mathcal{R}$ be rendering-only. For every old closed type A of role ρ, with $\rho \neq \mathsf{R}$ and $\mathsf{gradefree}(A)$, the extension $\mathfrak{S} + \mathcal{R}$ is term-conservative at A and reflects equality of old terms. In particular, at this observation boundary it cannot change inhabitation of such a type formed over $\mathfrak{S}$: the normal form of every closed term of A over $\mathfrak{S} + \mathcal{R}$ is a closed term of A over $\mathfrak{S}$. This is the book's key idea, that nothing may depend on a reading, as a theorem about extensions.*

Proof. Normalize a closed public extended $\vdash_\rho t : A$. By Lemma 10.2(i), $\mathsf{nf}(t)$ is old, proving term conservativity. If A is formed over $\mathfrak{S}$, the old normal form $\mathsf{nf}(t)$, which has type A by Theorem 8.5, has type A over $\mathfrak{S}$ by Proposition C.4; and every term over $\mathfrak{S}$ is a term over $\mathfrak{S} + \mathcal{R}$. So inhabitation does not change.

An old term contains no new atom and hence has exactly its base reductions. If two old terms are equal in the extension, confluence gives a common normal form, which is reached using only base reductions. Their equality is therefore reflected. □

Theorem 10.6 (Fresh-family conservativity)**.** *Let $\mathcal{F}$ be a fresh-witness-family module. It is term-conservative and equality-reflecting at every old closed grade-free type whose role is F or P.*

Proof. Term conservativity is Lemma 10.2(ii). Equality reflection is the same old-reduction argument as in Theorem 10.5. □

The protected-role qualification is exact. A fresh witness family is new vocabulary at role W, and the policy freely permits witnesses to occur in scripts and renderings. No conservativity claim is made there.

10.3 Evidence inhabitants

For a module $\mathcal{E}$, write $\mathsf{Event}_\mathcal{E}$ for its qualified event constants. Each $e \in \mathsf{Event}_\mathcal{E}$ belongs to one unique evidence entry and has one matching indexed origin owned by $\mathcal{E}$. A certificate pairs it with a fresh payload constant; an exact attestation pairs it with the old closed term named by the entry.

Theorem 10.7 (No silent enabling)**.** *Let $\mathcal{E}$ be an evidence-inhabitant module and let $\vdash_\rho t : A$ be closed and public in $\mathfrak{S} + \mathcal{E}$, where A is old and grade-free and $\rho \in \{\mathsf{F}, \mathsf{P}\}$. If $\mathsf{nf}(t)$ is not old, then*

$$\mathsf{events}(\mathsf{nf}(t)) \cap \mathsf{Event}_\mathcal{E} \neq \varnothing.$$

The extension still reflects judgmental equality of old terms.

Proof. By Lemma 10.2(iii), any new atom in $\mathsf{nf}(t)$ occurs below an old node whose normal event field is owned by $\mathcal{E}$. The definition of events therefore gives the displayed nonempty intersection. The module adds no reduction rule, so old equality is reflected as before. □

This theorem replaces the false claim that witness-only extension preserves old forms. It states the stronger property appropriate to conditional construction: the extension may enable an old gate, but the enabled result identifies the responsible module in normal form.

10.4 Composition

Theorem 10.8 (Module composition)**.** *If $\mathcal{M}$ is well formed over $\mathfrak{S}$ and $\mathcal{N}$ is well formed over $\mathfrak{S}+\mathcal{M}$, with disjoint qualified names, then $\mathcal{M};\mathcal{N}$ is well formed over $\mathfrak{S}$. Concatenation is associative. Principal effects, normalization, mediation, and record adequacy are preserved. A conservativity corollary is exported only when the combined suffix remains in the corresponding rendering-only or fresh-family class, and only at the grade-free observation boundary of Theorems 10.5 and 10.6.*

Proof. Signature formation is an ordered telescope, so the first claim and associativity are immediate. The effect algebra and policy are global and do not change under extension. Qualified freshness keeps evidence-entry ownership disjoint. The metatheorems depend only on well-formedness. Syntactic module classes are not closed under arbitrary concatenation, which is why their conservativity conclusions are recomputed rather than assumed. □

11 Erasure and provenance

11.1 Role erasure

Let $K_{\mathfrak{S}}$ be the role-free dependent $\Pi\Sigma$-calculus with a unit type, obtained from the same public families, ordinary primitives, and certificate payload constants. Exact attestations add no payload declaration. Event and origin constants from both evidence-entry forms map to unit. Each gate contributes an ordinary erased constructor

$$\overline{g} : |\Delta| \to |A| \to |B|.$$

Erasure removes role and grade annotations and is homomorphic on ordinary syntax. On provenance syntax it is defined by

$$\begin{array}{rclcrcl} |\mathsf{Event}| & = & \mathbf{1}, & & |\mathsf{Origin}_A(e,t)| & = & \mathbf{1}, \\ |\mathsf{Stamped}_\sigma(A)| & = & |A|, & & |\mathsf{stamp}(t,e,o)| & = & |t|, \\ |\mathsf{gate}_g(\vec{d};s)| & = & \overline{g}(|\vec{d}|;|s|), & & |\mathsf{node}_g(\vec{d};t,e)| & = & \overline{g}(|\vec{d}|;|t|). \end{array}$$

Event and origin constants erase to the sole unit term. The target contains the ordinary beta and projection rules and no record rule.

Theorem 11.1 (Erasure simulation)**.** *If $\mathfrak{S};\Gamma \vdash_\rho t : A$, then $|\Gamma| \vdash |t| : |A|$ in $K_{\mathfrak{S}}$. If $t \longrightarrow t'$, then either $|t| \longrightarrow |t'|$ or $|t| = |t'|$. Hence definitional equality is preserved by erasure.*

Proof. Induction on typing and reduction. Ordinary cases are homomorphic. Stamp typing erases to payload typing. The gate and node rules both erase to the typing rule of $\overline{g}$. Beta and projections reduce in the target; the record step maps both sides to the same erased constructor application. □

Erasure is intentionally non-faithful. Two normal nodes with different event constants may have identical erasures. This loss is not an unspecified semantic accident: provenance loss occurs at the displayed event, origin, stamp, gate, and node clauses. Erasure may additionally identify expressions that differ only in role or grade annotations; no claim of a complete kernel characterization is made here.

11.2 Provenance normal forms

For a closed protected term t, define

$$\mathsf{pnf}(t) = \mathsf{nf}(t), \qquad \mathsf{support}(t) = \mathsf{events}(\mathsf{nf}(t)).$$

Because private nodes are intrinsically typed by their gate result, a provenance normal form cannot contain a record at an unrelated result type.

Proposition 11.2 (Provenance stability)**.** *If closed protected terms t and u are definitionally equal, then*

$$\mathsf{pnf}(t) = \mathsf{pnf}(u) \quad \textit{and} \quad \mathsf{support}(t) = \mathsf{support}(u).$$

Their erasures may agree even when their provenance normal forms differ.

Proof. Confluence gives the same normal form for definitionally equal terms. Event support is a deterministic structural function on that normal form. The last claim follows from the event-forgetting erasure clauses. □

12 The book in the calculus

The core calculus is independent of a domain signature. This section declares the book's frames in it: forms with their construction history, installed presentations, interpretation witnesses, states with their legality, scripts with their action, and renderings [1]. The declarations are an instantiation schema rather than a formalization of the whole book. The octagonal-rosette module of section 12.2 and the interpretation-profile instance of section 12.4 are finite admitted modules whose derivations, reduction traces, extensions and erasure are worked out against the printed rules; section 12.6 writes the capture of section 5.1 in the calculus. All three are checked in Lean (table 8): their signatures are admitted and their derivations, effects and traces are the ones displayed.

12.1 Forms and construction history

Declare at role F

$$\vdash_{\mathsf{F}} \mathsf{FinCode}\ \ \mathsf{type}, \qquad A :_{\mathsf{F}} \mathsf{FinCode} \vdash_{\mathsf{F}} \mathsf{History}(A)\ \ \mathsf{type}$$

with ordinary constructors for empty, singleton, finite-index, and dependent assembly histories. Define

$$\vdash_{\mathsf{F}} \mathsf{Form}\ \ \mathsf{type}, \qquad \mathsf{Form} := \Sigma(A :_{\mathsf{F}} \mathsf{FinCode}).\mathsf{History}(A).$$

A form retains its carrier and its construction history, as the book's FORM does. All generic form constructors use grade 0 arguments and therefore add no special authority.

In the context $X :_{\mathsf{F}} \mathsf{Form}$, let $X :_{\mathsf{F}} \mathsf{Form} \vdash_{\mathsf{W}} \mathsf{AttachData}_k(X)\ \ \mathsf{type}$ contain an attaching shape, map, and boundary evidence. Attachment is a registered form gate

$$\mathsf{attach}_k : (X :_{\mathsf{F}} \mathsf{Form}); \mathsf{Stamped}_{\mathsf{W}}(\mathsf{AttachData}_k(X)) \Longrightarrow_{\mathsf{F}} \mathsf{Form}.$$

Its private normal constructor has the schematic form

$$\mathsf{node}_{\mathsf{attach}_k}(X; a, e),$$

where a and e have grade-1 edges. Carrier evaluation may erase to generic assembly and a finite quotient, while the form's normal form retains the attachment event. Quotient normalization itself is not a core RATT rule and requires a separate certified computational extension.

For a closed $X : \mathsf{Form}$ and an admitted certificate bundle

$$d = (a_d, e_d, o_d) : \mathsf{AttachData}_k(X),$$

the rules derive the public term and its record step

$$\vdash_{\mathsf{F}} t : \mathsf{Form}, \qquad t = \mathsf{gate}_{\mathsf{attach}_k}(X; \mathsf{stamp}(a_d, e_d, o_d)) \longrightarrow \mathsf{node}_{\mathsf{attach}_k}(X; a_d, e_d).$$

Before contraction its principal effect is

$$\mathbf{e}_{\mathsf{attach}_k} \oplus \mathbf{e}_X \oplus 1 \otimes (\mathbf{e}_{a_d} \oplus \mathbf{e}_{e_d} \oplus \mathbf{e}_{o_d});$$

after contraction the $\mathbf{e}_{o_d}$ component is discarded and the other components retain their prefixes. The reduct is normal when its fields are normal, and $e_d \in \mathsf{events}(\mathsf{nf}(t))$. This is the concrete Artforms instance of subject reduction with effect improvement and residual record adequacy.

12.2 A worked octagonal-rosette module

We now give one finite module rather than only a declaration schema. It uses an evidence extension to authorize an octagonal rosette inlay and a rendering-only extension to observe the result. All displayed declarations are instances of the rules in appendix A.

Start from the preceding Artforms signature and add ordinary constants

$$c_8 :_{\mathsf{F}} \mathsf{FinCode}, \qquad h_8 :_{\mathsf{F}} \mathsf{History}(c_8), \qquad X_8 :_{\mathsf{F}} \mathsf{Form}, \qquad X_8 \coloneqq (c_8, h_8).$$

The constants have family-headed result types, while X_8 is a derived pair. Add the witness family

$$\mathsf{RosetteSpec}_8 : (X :_{\mathsf{F}} \mathsf{Form}) \Rightarrow_{\mathsf{W}} \mathsf{type},$$

whose intended fields describe eight boundary anchors, cyclic order, and nonintersection evidence. The registered gate is

$$\mathsf{inlay}_8 : (X :_{\mathsf{F}} \mathsf{Form}); \mathsf{Stamped}_{\mathsf{W}}(\mathsf{RosetteSpec}_8(X)) \Longrightarrow_{\mathsf{F}} \mathsf{Form} \quad [\mathsf{node}_{\mathsf{inlay}_8}].$$

Its ordinary parameter has demand 0, its credential edge has grade 1, and its result Form is grade-free.

The evidence-inhabitant extension $\mathcal{E}_8$ is the single certificate bundle

$$d_8 = (a_8, e_8, o_8) : \mathsf{RosetteSpec}_8(X_8).$$

Before $\mathcal{E}_8$, the family $\mathsf{RosetteSpec}_8(X_8)$ has no constructor. The canonical-forms theorem therefore shows that it has no closed inhabitant. The extension adds exactly the payload, event, and aligned origin required by Sig-Certificate; it adds no reduction rule.

The following table is a finite admission certificate for the staged module.

Entry	Admission rule	Discharged condition
c_8, h_8	Sig-Primitive	closed, grade-free, family-headed form types
$\mathsf{RosetteSpec}_8$	Sig-Family	form dependency is ordinary into witnesses
inlay_8	Sig-Gate	form parameter demand 0, witness credential demand 1, protected grade-free result
d_8	Sig-Certificate	closed non-rendering family-headed payload; fresh common owner

Table 6: Admission certificate for the octagonal-rosette gate and its evidence extension.

The gate can be postponed behind a higher-order interface, but only at the positive grade demanded by the binder policy:

$$W_8 = \lambda^1(s :_{\mathsf{W}} \mathsf{Stamped}_{\mathsf{W}}(\mathsf{RosetteSpec}_8(X_8))).\mathsf{gate}_{\mathsf{inlay}_8}(X_8; s).$$

The body component of s is exactly 1, so maximal elaboration derives

$$\vdash_{\mathsf{F}} W_8 : \Pi^1(s :_{\mathsf{W}} \mathsf{Stamped}_{\mathsf{W}}(\mathsf{RosetteSpec}_8(X_8))).\mathsf{Form}.$$

This function type is intentionally not grade-free: W_8 is a suspended authorization and has not emitted a record. Applying it to the extension certificate produces the old protected result type Form, where the grade-free theorem applies:

$$\begin{aligned} T_8 &:= W_8\,\mathsf{stamp}(a_8, e_8, o_8) \\ &\longrightarrow_{\beta} \mathsf{gate}_{\mathsf{inlay}_8}(X_8; \mathsf{stamp}(a_8, e_8, o_8)) \\ &\longrightarrow^{\ell_8}_{\mathsf{record}} N_8 := \mathsf{node}_{\mathsf{inlay}_8}(X_8; a_8, e_8). \end{aligned}$$

All fields are constants or the normal pair X_8, so $N_8 = \mathsf{nf}(T_8)$. Writing $\xi_8 = \mathbf{e}_{c_8} \oplus \mathbf{e}_{h_8}$, the first two terms have effect

$$\mathbf{e}_{\mathsf{inlay}_8} \oplus \xi_8 \oplus 1 \otimes (\mathbf{e}_{a_8} \oplus \mathbf{e}_{e_8} \oplus \mathbf{e}_{o_8}),$$

and the node has the same effect without the o_8-component. The residual trace and support are

$$(\ell_8, e_8, e_8) \in \mathsf{Cross}^{\downarrow}(T_8 \longrightarrow^* N_8, N_8), \qquad e_8 \in \mathsf{events}(N_8).$$

Thus $\mathcal{E}_8$ is not term-conservative at Form: it enables the non-old normal form N_8. It satisfies no-silent-enabling exactly, since the enabling module's qualified event occurs in that form.

For a separate observation layer, extend by

$$\mathcal{V}_8 = \{\vdash_{\mathsf{R}} \mathsf{Canvas}_8 \;\; \mathsf{type}, \quad \mathsf{draw}_8 :_{\mathsf{R}} \Pi^0(X :_{\mathsf{F}} \mathsf{Form}).\mathsf{Canvas}_8\}.$$

This suffix is rendering-only over the already extended signature. Theorems 10.5 and 10.8 show that adding it neither creates another old protected form nor changes equality between existing forms.

Finally, erasure makes the operational loss of the certificate explicit:

$$|W_8| = \lambda s.\overline{\mathsf{inlay}}_8(|X_8|; s),$$
$$|T_8| \longrightarrow_\beta \overline{\mathsf{inlay}}_8(|X_8|; |a_8|) = |N_8|.$$

The source record step is the equality on the right, hence a stuttering erasure step; e_8 and o_8 are absent. As a concrete finite structural interpretation, take a vocabulary with binary relations Rim and Spoke and set

$$[\![|N_8|]\!]_8 = (\mathsf{Fin}(9), \{(i, 1 + (i \bmod 8)) \mid 1 \le i \le 8\}, \{(0, i) \mid 1 \le i \le 8\}).$$

This is the directed rim and eight spokes of a wheel. Its canonical finite table records the realized rosette, while the source normal form additionally records which certificate event authorized it.

12.3 Installed presentations

The book's PRESENTATION retains a selected order, seams, coordinates, cuts and sectors. A description offered to a constructor is witness data; once installed it is presentation, and the installation is a gate. For a selected order,

$$\vdash_{\mathsf{W}} \mathsf{OrderSpec}(F) \;\; \mathsf{type}, \qquad \vdash_{\mathsf{P}} \mathsf{Ordered}(F) \;\; \mathsf{type},$$

and the presentation gate is

$$\mathsf{installOrder} : (F :_{\mathsf{F}} \mathsf{Form}); \mathsf{Stamped}_{\mathsf{W}}(\mathsf{OrderSpec}(F)) \Longrightarrow_{\mathsf{P}} \mathsf{Ordered}(F).$$

The form F is an ordinary parameter. It is not recovered by projecting a presentation-role pair back into the form role. An artifact consists externally of the already constructed F and a term of $\mathsf{Ordered}(F)$; the normal form of the presentation records the installation event without pretending that the event constructed F.

The book's base cut of a wheel is this pattern: offered to the constructor that points the wheel it is a witness, and carried by the pointed wheel it is presentation [1, §1.9]. So is a supplied policy: witness data while it is supplied, presentation once a realizer carries it (section 5.2). Coordinates, seams, cuts and sectors use the same indexed pattern. A descriptor lives at W, the installed family at P, and installation is a presentation gate.

12.4 Interpretation witnesses and the raw/interpreted split

The book reads one raw artform in several ways without changing it: a palette laid on a clock by recurrence or by division, a template recognized in a canvas, a connection read through an invariant. The book's interpretation witness is a realizer paired with the certificate that the realizer produces an endpoint from the artform and obeys the law of a profile, and interpretation is a species of the witness role rather than a sixth role [1, Def. 1.9]. In $\mathsf{RATT}_{\mathrm{c}}$ that architecture is a declaration schema at role W, and the rules fix exactly what the schema may package.

An *interpretation profile* $\mathcal{C}$ over a raw type Art, which is Form or a declared form-role family, consists of an endpoint family and two witness families,

$$\Theta_{\mathcal{C}} : (F :_{\mathsf{F}} \mathsf{Art}) \Rightarrow_{\rho_{\mathcal{C}}} \mathsf{type}, \qquad \mathcal{R}_{\mathcal{C}} : (F :_{\mathsf{F}} \mathsf{Art},\ \theta :_{\rho_{\mathcal{C}}} \Theta_{\mathcal{C}}(F)) \Rightarrow_{\mathsf{W}} \mathsf{type},$$
$$\mathcal{L}_{\mathcal{C}} : (F :_{\mathsf{F}} \mathsf{Art},\ \theta :_{\rho_{\mathcal{C}}} \Theta_{\mathcal{C}}(F),\ \ell :_{\mathsf{W}} \mathcal{R}_{\mathcal{C}}(F,\theta)) \Rightarrow_{\mathsf{W}} \mathsf{type},$$

with $\rho_{\mathcal{C}} \neq \mathsf{R}$, together with ordinary constructors for the two witness families. Every telescope edge into W has demand 0 in table 2, so Sig-Family admits both families. The interpretation witness type is the derived homogeneous sum

$$\mathsf{IntWit}_{\mathcal{C}}(F,\theta) \coloneqq \Sigma(\ell :_{\mathsf{W}} \mathcal{R}_{\mathcal{C}}(F,\theta)).\, \mathcal{L}_{\mathcal{C}}(F,\theta;\ell),$$

formed by Ty-Sigma at role W. Its first projection is the supplied realizer and its second the law certificate; the source F, the profile, and the endpoint θ remain indices, and the source enters only through ordinary form edges. Because both components are witness-family applications, (WitnessData) gives $\mathsf{wdata}(\mathsf{IntWit}_{\mathcal{C}}(F,\theta))$: a closed public inhabitant $I = (\ell, c)$ is exactly attestable by Sig-Attest without a fresh payload constant.

Three features of the core determine what the schema does not contain.

(i) There is no internal type of profiles or endpoints, and dependent sums are homogeneous in role. The book's semantic fibre, the sum over the available profiles and their endpoints of $\mathsf{IntWit}_{\mathcal{C}}(F,\theta)$, is therefore not a $\mathsf{RATT}_{\mathrm{c}}$ type. It is the external family of witness types indexed by signature entries and endpoint terms. For the same reason the layer pairing a raw F with one selected witness is not a type: as for $\mathsf{Ordered}(F)$ in section 12.3, an artifact with a selected reading consists externally of the already constructed F and a term of $\mathsf{IntWit}_{\mathcal{C}}(F,\theta)$. The book's fibre stands beside F and selects nothing, and choosing one witness is a further step; here both are consequences of the universe-free boundary of section 6.2.

(ii) An endpoint cannot be a rendering. A rendering-role index has demand ∞ into W, so a profile whose endpoints are graphics, scores, or observed values fails policy validation. A value read off an artform enters only as a closed literal or meta-level finite index, which carries no declaration atom.

(iii) Forming a witness installs nothing. A selected inhabitant becomes an installed presentation only through a presentation gate

$$\mathsf{install}_{\mathcal{C}} : (F :_{\mathsf{F}} \mathsf{Art},\ \theta :_{\rho_{\mathcal{C}}} \Theta_{\mathcal{C}}(F));$$
$$\mathsf{Stamped}_{\mathsf{W}}(\mathsf{IntWit}_{\mathcal{C}}(F,\theta)) \Longrightarrow_{\mathsf{P}} \mathsf{Interp}_{\mathcal{C}}(F,\theta) \quad [\mathsf{node}_{\mathsf{install}_{\mathcal{C}}}],$$

where $\mathsf{Interp}_{\mathcal{C}}$ is a presentation-role family over the same telescope. Sig-Gate requires $\rho_{\mathcal{C}} \in \{\mathsf{F}, \mathsf{P}\}$, since a parameter has demand 0 into the result role and a witness- or script-role parameter has demand 1 into P; the parameter and payload types are grade-free; and the result does not depend on the payload. A certificate bundle cannot deliver the credential, because Sig-Certificate requires a family-headed payload type and $\mathsf{IntWit}_{\mathcal{C}}(F,\theta)$ is a sum. The route into the gate is exact attestation of an already checked pair.

Proposition 12.1 (Interpretation profiles are conservative)**.** *Let $\mathcal{M}_{\mathcal{C}}$ add, over a signature in which* Art *and $\Theta_{\mathcal{C}}$ are old, exactly the two witness families $\mathcal{R}_{\mathcal{C}}$, $\mathcal{L}_{\mathcal{C}}$ and ordinary constructors headed by them. Then $\mathcal{M}_{\mathcal{C}}$ is a fresh-witness-family module, and by Theorem 10.6 it is term-conservative and equality-reflecting at every old closed grade-free type of role* F *or* P*. In particular a new way of reading F creates no raw artform, identifies none, and installs no presentation.*

Proof. The suffix consists of witness-role families and constructors headed by them, which is the fresh-witness-family class of section 10; the rest is Theorem 10.6. □

The hypothesis that $\Theta_{\mathcal{C}}$ is old is exact. A profile that also declares a new form- or presentation-role endpoint family leaves the class, and no conservativity claim is exported for it.

Proposition 12.2 (Attested installation is traceable)**.** *Let I be a closed public term of $\mathsf{IntWit}_{\mathcal{C}}(F,\theta)$ in a signature containing $\mathcal{M}_{\mathcal{C}}$ and $\mathsf{install}_{\mathcal{C}}$, and let $\mathcal{E}_q$ be the exact attestation $q@I = (e_q, o_q)$. Then $\mathcal{E}_q$ is an evidence-inhabitant module, and the public term*

$$\mathsf{gate}_{\mathsf{install}_{\mathcal{C}}}(F,\theta;\mathsf{stamp}(I,e_q,o_q))$$

contracts to $\mathsf{node}_{\mathsf{install}_{\mathcal{C}}}(F,\theta;I,e_q)$, with the exact pair retained and e_q in its event support; and, by Theorem 10.7, every closed public term of an old grade-free protected type whose normal form is not old contains an event owned by $\mathcal{E}_q$.

Proof. The attestation is admitted by Sig-Attest, whose premises were verified above, and it adds neither family, primitive, nor gate, so the module is evidence-inhabitant. The contraction is one instance of the record rule, and Lemma 9.5 carries e_q to the normal form. The last claim is Theorem 10.7. □

We now give one finite instance, worked out against the printed rules in the same way as section 12.2. Extend the Artforms signature with an endpoint family and closed constants

$$\mathsf{Tap} : (F :_{\mathsf{F}} \mathsf{Form}) \Rightarrow_{\mathsf{F}} \mathsf{type}, \qquad c_Q :_{\mathsf{F}} \mathsf{FinCode}, \quad h_Q :_{\mathsf{F}} \mathsf{History}(c_Q), \quad Q \coloneqq (c_Q, h_Q),$$
$$\theta_{\mathrm{rec}}, \theta_{\mathrm{div}} :_{\mathsf{F}} \mathsf{Tap}(Q),$$

where Q is intended as a palette, $\mathsf{Tap}(Q)$ as the tapestries laid from it, and the two constants as the tapestries obtained by laying the palette by recurrence and by division. A laid tapestry is a form, as the book's tapestries are, so the endpoints are form-role data. The recurrence profile module $\mathcal{M}_{\mathrm{rec}}$ consists of

$$\mathsf{Lay}_{\mathrm{rec}} : (F :_{\mathsf{F}} \mathsf{Form},\ \theta :_{\mathsf{F}} \mathsf{Tap}(F)) \Rightarrow_{\mathsf{W}} \mathsf{type},$$
$$\mathsf{Obeys}_{\mathrm{rec}} : (F :_{\mathsf{F}} \mathsf{Form},\ \theta :_{\mathsf{F}} \mathsf{Tap}(F),\ \ell :_{\mathsf{W}} \mathsf{Lay}_{\mathrm{rec}}(F,\theta)) \Rightarrow_{\mathsf{W}} \mathsf{type},$$
$$\ell_{\mathrm{rec}} :_{\mathsf{W}} \mathsf{Lay}_{\mathrm{rec}}(Q,\theta_{\mathrm{rec}}), \qquad c_{\mathrm{rec}} :_{\mathsf{W}} \mathsf{Obeys}_{\mathrm{rec}}(Q,\theta_{\mathrm{rec}},\ell_{\mathrm{rec}}),$$

whose intended fields are the rule that advances every row by one phase per tick and the check that its evaluation is θ_{rec}; $\mathcal{M}_{\mathrm{div}}$ is the same with div and the rule that holds a phase for a whole bar. Both modules are fresh-witness-family, so Proposition 12.1 applies: after either or both, the closed form terms c_Q, h_Q, Q and every old closed term of type Form are the same old terms with the same equalities. The two witnesses

$$I_{\mathrm{rec}} \coloneqq (\ell_{\mathrm{rec}}, c_{\mathrm{rec}}) : \mathsf{IntWit}_{\mathrm{rec}}(Q,\theta_{\mathrm{rec}}), \qquad I_{\mathrm{div}} \coloneqq (\ell_{\mathrm{div}}, c_{\mathrm{div}}) : \mathsf{IntWit}_{\mathrm{div}}(Q,\theta_{\mathrm{div}})$$

are closed normal terms of two witness-role types over the one form constant Q. Nothing relates θ_{rec} to θ_{div}: the core has no identity type and no user equations, and by Theorem 8.14 two distinct constants are distinct normal forms. "The same palette, laid twice" is thus literally two readings beside one unchanged Q.

Installation of the recurrence reading is the gate module

$$\mathsf{Laid}_{\mathrm{rec}} : (F :_{\mathsf{F}} \mathsf{Form},\ \theta :_{\mathsf{F}} \mathsf{Tap}(F)) \Rightarrow_{\mathsf{P}} \mathsf{type},$$
$$\mathsf{installLaying}_{\mathrm{rec}} : (F :_{\mathsf{F}} \mathsf{Form},\ \theta :_{\mathsf{F}} \mathsf{Tap}(F));$$
$$\mathsf{Stamped}_{\mathsf{W}}(\mathsf{IntWit}_{\mathrm{rec}}(F,\theta)) \Longrightarrow_{\mathsf{P}} \mathsf{Laid}_{\mathrm{rec}}(F,\theta) \quad [\mathsf{node}_{\mathsf{installLaying}_{\mathrm{rec}}}],$$

and the evidence-inhabitant extension is the single exact attestation $\mathcal{E}_{\mathrm{rec}} = \{q@I_{\mathrm{rec}} = (e_{\mathrm{rec}}, o_{\mathrm{rec}})\}$. Table 7 is the admission certificate.

Entry	Admission rule	Discharged condition
$\mathsf{Tap};\ \theta_{\mathrm{rec}}, \theta_{\mathrm{div}}$	Sig-Family; Sig-Primitive	form dependency ordinary into form; closed, grade-free, family-headed form types
$\mathsf{Lay}_{\mathrm{rec}}$, $\mathsf{Obeys}_{\mathrm{rec}}$	Sig-Family	form and witness dependencies ordinary into witnesses
ℓ_{rec}, c_{rec}	Sig-Primitive	closed, grade-free, headed by the new witness families
$\mathsf{IntWit}_{\mathrm{rec}}(Q, \theta_{\mathrm{rec}})$	Ty-Sigma	homogeneous at W; witness data
$\mathsf{Laid}_{\mathrm{rec}}$; $\mathsf{installLaying}_{\mathrm{rec}}$	Sig-Family; Sig-Gate	form parameters demand 0 into presentation; witness credential demand 1; protected grade-free result independent of the payload
$q@I_{\mathrm{rec}}$	Sig-Attest	closed public witness-data payload; grade-free; no payload constant; fresh common owner

Table 7: Admission certificate for the recurrence interpretation profile, its installation gate, and its exact attestation.

The public term and its record step are

$$T_{\mathrm{rec}} := \mathsf{gate}_{\mathsf{installLaying}_{\mathrm{rec}}}(Q, \theta_{\mathrm{rec}}; \mathsf{stamp}(I_{\mathrm{rec}}, e_{\mathrm{rec}}, o_{\mathrm{rec}}))$$
$$\longrightarrow^{\ell_{\mathrm{rec}}}_{\mathsf{record}} N_{\mathrm{rec}} := \mathsf{node}_{\mathsf{installLaying}_{\mathrm{rec}}}(Q, \theta_{\mathrm{rec}}; I_{\mathrm{rec}}, e_{\mathrm{rec}}).$$

Writing $\xi_Q = \mathbf{e}_{c_Q} \oplus \mathbf{e}_{h_Q}$, the effect of T_{rec} is

$$\mathbf{e}_{\mathsf{installLaying}_{\mathrm{rec}}} \oplus \xi_Q \oplus \mathbf{e}_{\theta_{\mathrm{rec}}} \oplus 1 \otimes (\mathbf{e}_{\ell_{\mathrm{rec}}} \oplus \mathbf{e}_{c_{\mathrm{rec}}} \oplus \mathbf{e}_{e_{\mathrm{rec}}} \oplus \mathbf{e}_{o_{\mathrm{rec}}}),$$

and the node has the same effect without the o_{rec}-component: subject reduction with effect improvement. All fields are constants or normal pairs, so $N_{\mathrm{rec}} = \mathsf{nf}(T_{\mathrm{rec}})$, $e_{\mathrm{rec}} \in \mathsf{events}(N_{\mathrm{rec}})$, and $(\ell_{\mathrm{rec}}, e_{\mathrm{rec}}, e_{\mathrm{rec}}) \in \mathsf{Cross}^{\downarrow}(T_{\mathrm{rec}} \longrightarrow N_{\mathrm{rec}}, N_{\mathrm{rec}})$. The exact pair I_{rec} is retained in the node, and no projection equation was needed to recover its realizer. A second attestation $q'@I_{\mathrm{rec}} = (e'_{\mathrm{rec}}, o'_{\mathrm{rec}})$ installs the same reading with a different event: the two nodes are distinct source terms with the same erasure

$$|N_{\mathrm{rec}}| = \overline{\mathsf{installLaying}}_{\mathrm{rec}}(|Q|, |\theta_{\mathrm{rec}}|; |I_{\mathrm{rec}}|),$$

which forgets e_{rec} and o_{rec} as Theorem 11.1 prescribes. Installing the division reading is the same derivation with div, and its node carries e_{div}. A finite structural interpretation may then denote $|N_{\mathrm{rec}}|$ and $|N_{\mathrm{div}}|$ by two different tables over the same carrier, while only the source normal forms record which reading and which attestation authorized each.

12.5 Frames, states and scripts

A frame of the book is a carrier, its carried structure, the space of legal states, the allowed observations and the allowed moves. Its first two parts are the forms and installed presentations above. For a presentation A declare

$$\vdash_{\mathsf{W}} \mathsf{Table}(A)\ \ \mathsf{type}, \qquad \mathsf{Legal}_A : (s :_{\mathsf{W}} \mathsf{Table}(A)) \Rightarrow_{\mathsf{W}} \mathsf{type},$$
$$\mathsf{State}(A) := \Sigma(s :_{\mathsf{W}} \mathsf{Table}(A)).\, \mathsf{Legal}_A(s).$$

A table is a choice of a value at every position of A, and a state is a table with the evidence that it is legal. The sum is homogeneous at W, so Ty-Sigma forms it: the book's formation rule for the state space is a witness-role sum, as section 2.2 reads it. The presentation A enters only as an index, through an ordinary edge from P into W.

The sum keeps the evidence, and the calculus has no rule that identifies two proofs. Two legality proofs $\ell_0, \ell_1 : \mathsf{Legal}_A(t_0)$ of one table give the states (t_0, ℓ_0) and (t_0, ℓ_1), distinct normal forms that are not definitionally equal. The book counts a legal table once (section 2.2). In the calculus that count is an observation of states through their tables: the tables of the states are exactly the legal tables, and counting the states themselves gives the book's count exactly when no table has two legality proofs. A signature that needs the book's count declares legality with at most one proof per table, or counts through fst; the core adds no proof-irrelevance rule.

For presentations A and B declare

$$\vdash_{\mathsf{S}} \mathsf{Script}(A,B)\ \ \mathsf{type}, \qquad \mathsf{id}_A :_{\mathsf{S}} \mathsf{Script}(A,A),$$
$$\mathsf{comp} :_{\mathsf{S}} \Pi^0(f :_{\mathsf{S}} \mathsf{Script}(A,B)).\, \Pi^0(g :_{\mathsf{S}} \mathsf{Script}(B,C)).\, \mathsf{Script}(A,C),$$
$$\mathsf{act}_{\mathsf{T}} :_{\mathsf{W}} \Pi^0(f :_{\mathsf{S}} \mathsf{Script}(A,B)).\, \Pi^0(t :_{\mathsf{W}} \mathsf{Table}(A)).\, \mathsf{Table}(B),$$
$$\mathsf{act}_{\mathsf{L}} :_{\mathsf{W}} \Pi^0(f :_{\mathsf{S}} \mathsf{Script}(A,B)).\, \Pi^0(t :_{\mathsf{W}} \mathsf{Table}(A)).\, \Pi^0(\ell :_{\mathsf{W}} \mathsf{Legal}_A(t)).\, \mathsf{Legal}_B(\mathsf{act}_{\mathsf{T}}\, f\, t).$$

Script generators are further primitives at S. Composition is typed at the middle presentation, as in the book. The action of a declared script on a state is the derived term

$$\mathsf{act}\, f\, p \coloneqq (\mathsf{act}_{\mathsf{T}}\, f\, (\mathsf{fst} p),\ \mathsf{act}_{\mathsf{L}}\, f\, (\mathsf{fst} p)\, (\mathsf{snd} p)) : \mathsf{State}(B),$$

which sends a state of the source to a state of the target and keeps it legal: a declared script is total on legal states. Its result is witness data, since the codomain decides. Every edge in these declarations has demand 0, since $D(\mathsf{S},\mathsf{W}) = D(\mathsf{W},\mathsf{W}) = 0$, so they are ordinary primitives and need no gate. A move changes the instance and not the frame.

The core treats script equality as judgmental equality of the freely generated syntax only. Two scripts may have the same action and remain two scripts, as the book requires: that their actions agree is itself witness data, an inhabitant of a family $\mathsf{SameAction}(f,g)$ at W indexed by the two scripts. Fusion, commutation or cancellation equations belong to a separate typed setoid, after a proof that their congruence reflects old equality; they do not participate in dependent conversion.

A guarded move is total on its admitted input, including the needed evidence, and not on every state of the frame [1, §1.9]. It has its own type, and its action takes the evidence that it applies:

$$\vdash_{\mathsf{S}} \mathsf{Guarded}(A,B)\ \ \mathsf{type}, \qquad \mathsf{Applicable} : (f :_{\mathsf{S}} \mathsf{Guarded}(A,B),\ s :_{\mathsf{W}} \mathsf{State}(A)) \Rightarrow_{\mathsf{W}} \mathsf{type},$$
$$\mathsf{actG}_{\mathsf{T}} :_{\mathsf{W}} \Pi^0(f :_{\mathsf{S}} \mathsf{Guarded}(A,B)).\, \Pi^0(s :_{\mathsf{W}} \mathsf{State}(A)).\, \Pi^0(c :_{\mathsf{W}} \mathsf{Applicable}(f,s)).\, \mathsf{Table}(B),$$
$$\mathsf{actG}_{\mathsf{L}} :_{\mathsf{W}} \Pi^0 f.\, \Pi^0 s.\, \Pi^0 c.\, \mathsf{Legal}_B(\mathsf{actG}_{\mathsf{T}}\, f\, s\, c),$$
$$\mathsf{actG}\, f\, s\, c \coloneqq (\mathsf{actG}_{\mathsf{T}}\, f\, s\, c,\ \mathsf{actG}_{\mathsf{L}}\, f\, s\, c) : \mathsf{State}(B).$$

$\mathsf{Applicable}$ is admitted because both edges are ordinary into W. A guarded move is not a declared script, so act does not apply to it: the guard is enforced by typing, and a guarded move acts only together with a certificate that it applies. Requirement (R4) appears here twice: the certificate names the move, and the guarded action takes its certificate. The step from witnesses and scripts back into the frame is gated. A position that becomes the fixed start of a puzzle, or a history that becomes a game score, enters a form or a presentation only through a gate, and its normal form records the event that authorized it. For a score,

$$\mathsf{note} : (T :_{\mathsf{F}} \mathsf{Score});\ \mathsf{Stamped}_{\mathsf{W}}(\mathsf{Move}) \Longrightarrow_{\mathsf{F}} \mathsf{Score}, \qquad \mathsf{mv} :_{\mathsf{W}} \Pi^0(f :_{\mathsf{S}} \mathsf{Guarded}(A,A)).\, \mathsf{Move},$$

where Move is a closed witness family of move records and $\mathsf{mv}\, f$ is the record of the move f. An exact attestation of $\mathsf{mv}\, f$ supplies the event that the move was made. Each contraction retains the move and its event, so a score in normal form is the list of the moves made with their events. This is the book's run, a trajectory together with the trace of the moves made, with the trace as the record of the gates.

12.6 One capture in the calculus

The capture of table 4 is a closed derivation in such a signature. Let $\mathsf{Board} \coloneqq (c_B, h_B) : \mathsf{Form}$ be the board, built from constants as X_8 is in section 12.2; $A :_{\mathsf{P}} \mathsf{Addr}(\mathsf{Board})$ the addressing; $s_0 \coloneqq (t_0, \ell_0) : \mathsf{State}(A)$ the position, with $t_0 :_{\mathsf{W}} \mathsf{Table}(A)$ and $\ell_0 :_{\mathsf{W}} \mathsf{Legal}_A(t_0)$; and $\mathsf{nxe5} :_{\mathsf{S}} \mathsf{Guarded}(A, A)$ the capture. The squares White bears on are a derived state,

$$\mathsf{attack} :_{\mathsf{W}} \Pi^0(s :_{\mathsf{W}} \mathsf{State}(A)).\, \mathsf{Table}_{\mathsf{Bool}}(A),$$

a value on every square, and the certificate of the capture is

$$c \coloneqq \mathsf{check}\, \mathsf{nxe5}\, s_0\, (\mathsf{attack}\, s_0) : \mathsf{Applicable}(\mathsf{nxe5}, s_0),$$
$$\mathsf{check} :_{\mathsf{W}} \Pi^0 f.\, \Pi^0 s.\, \Pi^0(b :_{\mathsf{W}} \mathsf{Table}_{\mathsf{Bool}}(A)).\, \mathsf{Applicable}(f, s).$$

The position after the capture is $\mathsf{actG}\, \mathsf{nxe5}\, s_0\, c$. Its effect has the component 0 at $\mathsf{actG}_{\mathsf{T}}$, $\mathsf{actG}_{\mathsf{L}}$, $\mathsf{nxe5}$, t_0, ℓ_0, check and attack, and ∞ elsewhere: it depends on the position, on the capture and on the certificate. The engine's evaluation is

$$\mathsf{eval} :_{\mathsf{R}} \Pi^0(s :_{\mathsf{W}} \mathsf{State}(A)).\, \mathsf{Num}, \qquad \vdash_{\mathsf{R}} \mathsf{Num}\ \ \mathsf{type}\ .$$

A certificate family indexed by $\mathsf{eval}\, s_0$ cannot be formed: its formation edge from a rendering into W has demand ∞, so Sig-Family rejects it. A rule that consults a reading is not merely discouraged; it is not a well-formed declaration.

The derivations carry the dependencies of table 4. Its edges into the squares White bears on, the certificate, the position after the capture, the evaluation and the diagram are term effects: each of these items has grade 0 at every atom of the items it depends on. The frame enters through types. The addressing is declared at $\mathsf{Addr}(\mathsf{Board})$, whose formation effect has grade 0 at the board's constants, and the types of the position, of the squares White bears on and of the capture, $\mathsf{State}(A)$, $\mathsf{Table}_{\mathsf{Bool}}(A)$ and $\mathsf{Guarded}(A, A)$, have grade 0 at A. So every witness item, and the capture, depends on the addressing through its type; the table records that dependency once, at the position. The score $\mathsf{note}(T_0; \mathsf{stamp}(\mathsf{mv}\, \mathsf{nxe5}, e, o))$ has grade 1 at the capture and at its record, and one record step contracts it to the node $\mathsf{note}_0(T_0; \mathsf{mv}\, \mathsf{nxe5}, e)$, which retains the move and its event. A form or a presentation uses the capture at grade at least 1, and a primitive that would compute a score from a guarded move is rejected at every grade. These are the three statements of Proposition 5.1 for this signature: it is admitted, and its readback and silent-score variants are not.

The signature is schematic. Its primitives postulate the actions, the attack map and the check, so it shows how the roles and grades of table 4 are carried by derivations; it does not implement the rules of chess.

12.7 Rendering firewall

A rendering is a value at a rendering-role type, and a readout is a script or function whose values are renderings. Declare

$$\vdash_{\mathsf{R}} \mathsf{Graphic}\ \ \mathsf{type}, \qquad r :_{\mathsf{R}} \Pi^0(s :_{\mathsf{W}} \mathsf{State}(A)).\mathsf{Graphic}.$$

The dependency of the drawing on the state is ordinary. Conversely, a function type with a graphic domain and a non-rendering result fails policy validation unless the domain edge is

∞; no primitive may declare such an edge, stamps cannot contain rendering payloads, and gates cannot target rendering data. Thus replacing the renderer cannot alter an applicability derivation, which is Theorem 10.5 for this signature and Theorem 3.8 for its values.

A policy that chooses among applicable moves is witness data while it is supplied. It may depend on states and certificates, but not on a rendering; a scheduler that inspects a drawing to choose a move is outside the calculus. Installed in a realizer, the policy becomes presentation through a gate, as in section 12.3.

The protected observation types used above—Form, Score, $\mathsf{Ordered}(F)$, and every admitted gate result—are grade-free. Consequently the adequacy and module theorems apply to completed forms, scores and installed presentations. A higher-order scheduler interface whose type itself contains grade 1 or ∞ is intentionally outside that observation boundary until it is applied to a closed implementation.

12.8 Finite structural semantics

For a fixed finite relational vocabulary L, an Artforms signature may supply a total interpretation

$$[\![-]\!]_L : |\mathsf{Form}| \longrightarrow \mathsf{FinStr}_L$$

on erased normal forms. Canonical labeling over all bijections to $\mathsf{Fin}(n)$ decides isomorphism of the resulting finite structures: two structures on $\mathsf{Fin}(n)$ are isomorphic exactly when their least tables over all relabellings agree. The map factors as

$$\text{RATT term} \longrightarrow \text{provenance normal form} \xrightarrow{\mathsf{erase}} \text{finite structure} \longrightarrow \text{canonical table}.$$

The first arrow preserves gate events; the second intentionally forgets them. Different construction histories may therefore denote the same finite structure without becoming judgmentally equal.

13 Relationship to existing work

The textbook. The five roles, their readings, the policy in prose and its key idea that nothing may depend on a reading are the book's [1, §1.9]. So are the frame and its instance, the four useful words that are not roles, the rules of inference read as a second discipline beside the roles [1, §1.11], and the count of histories that holds moves and not proofs [1, §7.4]. This paper adds the levels, the uniqueness and path closure of the policy, the separation theorem, the decision that a state is witness data, and the proofs. The book does not cite this paper; the repository that holds the Lean proofs of the book's theorems also holds this paper and its Lean library [2].

Graded and multimodal dependent type theory. Multimodal dependent type theory parameterizes syntax by modes, modalities, and transformations and proves uniform canonicity and normalization results [8, 9]. Graded modal dependent type theory tracks variable use in terms and types and establishes normalization, decidable equality, and erasure properties [10]. $\mathsf{RATT}_{\mathrm{c}}$ does not claim roles or graded substitution as new in isolation. Its grade algebra measures the least mediation over occurrence paths, and its generative restriction ties the only positive primitive grade to a record-emitting reduction rule.

Phase separation and noninterference. Modal type theories connect phase distinctions with information-flow noninterference [11]. Type Theory with Erasure gives a structural phase distinction and conservativity over Martin-Löf type theory [14]. Abstraction Functions as Types derives modularity from a phase distinction in univalent dependent type theory [15]. RATT's rendering theorem is a small generative noninterference result, while its evidence theorem deliberately permits a declassification path and requires an intrinsic record.

Dependency security and declassification. The Dependency Core Calculus internalizes protection labels in types and proves noninterference properties for dependency [12]; systematic accounts of declassification distinguish what information is released, by whom, where, and when [13]. The four levels of section 2.4 order the roles as a lattice of labels orders data, and the firewall is a noninterference theorem. But $\mathsf{RATT}_{\mathrm{c}}$ does not claim a confidentiality theorem, and its roles are not security levels: a gated crossing is an installation that is recorded, not a release of a secret. Its narrower connection is controlled release into a protected constructor: the release edge is generative, and a contributing release leaves a proof-relevant record in the normal form. The grade-free boundary in remark 10.3 is necessary because an unapplied higher-order interface can postpone such a release without yet producing a record.

Provenance. Tracing calculi for functional programs use replayable traces and support multiple provenance views [16]. RATT differs by making a record constructor part of the normal form of a protected dependent term and by proving that the static grade-1 path and dynamic event arise from the same gate declaration.

Modular signatures. Generalized algebraic theories and categories with families provide standard accounts of dependent signatures, while modular presentations of type theory study when subtheory derivations remain in a smaller signature [7, 17]. RATT selects a deliberately generative fragment whose extension classes have different, syntactically checkable theorems.

Mechanization. Graded modal dependent type theory with a universe and erasure has been formalized in Agda [10]. Here both parts are formalized in Lean 4 with Mathlib [3, 4]: the role discipline, and the calculus with its metatheory, decision procedure and examples (section 14).

14 Theorem status and limitations

Every result of this paper is checked in Lean 4 with Mathlib, in the library `ArtformRoles` of the repository [2]: the first part in the namespace `Artform.Roles` (table 5) and the calculus in `Artform.Roles.Calculus` (table 8). The proofs use only the axioms `propext`, `Classical.choice` and `Quot.sound`, and the normalizer and the checker are computable functions. The theorems about the calculus presuppose a well-formed signature and, where they mention a context, a well-formed context; the Lean statements carry these hypotheses. Derivations are data in Lean, so the paths of a derivation (Lemma 8.1) are defined on them, and conversion is read as in section 7.

What the mechanization changed. Checking the calculus changed two rules and corrected or narrowed six claims of an earlier version of this paper.

(1) Sig-Gate requires a grade-free interface, not only a grade-free result (remark 10.4).

(2) Sig-Family requires every telescope entry to have demand 0 into the family's role; the text assumed it.

(3) The event support of a node includes the events inside its event field (section 7).

(4) Lemma 10.1(i) said that a new rendering primitive has a new rendering family as ultimate head; a rendering primitive may construct an old rendering family, and only its role keeps it out.

(5) Theorem 8.16 and Proposition C.2 hold on the bidirectional fragment, which contains every well-typed normal form (remark C.3).

Result	Lean name
Lemma 6.2	`Deriv.role_unique`
Lemma 8.1	`Deriv.eff_eq_pathEff`
Theorem 8.2	`Deriv.effect_unique`, `lam_grade_iff`, `lam_max_grade`
Lemma 8.3 and Theorem 8.4	`Deriv.inst_tm`, `Deriv.inst_ty`, `Deriv.subst`
Theorem 8.5	`Deriv.subject_reduction`, `Deriv.subject_reduction_ty`
Lemma 8.6	`gskel_conv`, `skel_conv`
Lemmas 8.7 to 8.9	`red_sn`, `red_step`, `red_gate`, `red_node`, `Deriv.fundamental`
Theorem 8.10	`Deriv.sn_tm`, `Deriv.sn_ty`
Lemma 8.11	`Step.subst`, `Steps.subst_arg`
Lemma 8.12 and Theorem 8.13	`Steps.local_confluent`, `Steps.confluent`
Theorem 8.14 and Corollary 8.15	`canonical_stamped`, `canonical_event`, `canonical_origin`, `canonical_lam`, `canonical_protected_neutral`, `canonical_credential`
Theorem 8.16	`conv_iff_nf_eq`, `Deriv.decConvTy`, `Deriv.decConvTm`, `decTyping`, `decFormed`, `decSigWF`, `decCtxWF`, `Deriv.origin_nonrendering`
Theorem 9.1	`Deriv.mediation`, `Deriv.mediation_steps`
Lemmas 9.2 to 9.4	`positive_function`, `Deriv.free_outside_nodes`, `Deriv.gradefree_mediation`
Lemmas 9.5 and 9.6 and Theorem 9.7	`record_persistence`, `node_ancestry`, `gate_adequacy`
Lemma 10.1	`old_head`, `stamp_entry`, `origin_protected`
Lemma 10.2	`extension_support_conservative`, `extension_support_evidence`
Remark 10.3	`GradeFreedom.Remark103.rendering_boundary`, `evidence_boundary`, `extension_support_needs_gradefree`, `no_silent_enabling_needs_gradefree`
Remark 10.4	$\texttt{SigWF}_0$, $\texttt{SigWF.toSigWF}_0$, `GradeFreedom.Remark104.rendering_gate`, `evidence_gate`, `extension_support_fails_old_rule`, `no_silent_enabling_fails_old_rule`
Theorems 10.5 and 10.6	`rendering_noninterference`, `rendering_inhabitation`, `fresh_family_conservative`
Theorems 10.7 and 10.8	`no_silent_enabling`, `module_composition`
Theorem 11.1 and Proposition 11.2	`Deriv.erase`, `Step.erase`, `provenance_stable`
Section 12.2	`Book.Rosette.trace`, `Book.Rosette.effects`, `Book.Rosette.record_trace`, `Book.Rosette.enabling`, `Book.Rosette.erasure`
Propositions 12.1 and 12.2	`Book.profile_conservative`, `Book.attested_installation`
interpretation instance	`Book.Interp.install`, `Book.Interp.second_attestation`, `Book.Interp.two_readings`
Sections 12.5 and 12.6	`Book.Chess.sig_wf`, `next_typed`, `guarded_not_script`, `effects_match_table`, `frame_types`, `score_typed`, `firewall`, `silent_score`, `two_legal_states`; `card_states_eq_iff`
finite structural semantics	`FinStr.iso_iff_canon_eq`, `FinStr.decIso`
Proposition C.2 and remark C.3	`complete`, `normal_fragment`, `checkSig_complete`, `checkCtx_complete`, `Book.Chess.checker_incomplete`
Proposition C.4	`checker_strengthen`, `Derivable.strengthen_ty`, `Derivable.strengthen`

Table 8: The results of the second part in the Lean library `ArtformRoles`, namespace `Artform.Roles.Calculus`; `card_states_eq_iff` is in `Artform.Roles`.

(6) Theorem 4.5(ii) holds under counting that is uniform by role (remark 4.6).

(7) In the capture, the guarded move acts through actG, which takes its certificate, and the claim that the position after has grade 0 at the declarations of A and the board holds for its type, not for its term (section 12.6).

(8) A state keeps its legality proof; the book's count is a count through the table (section 12.5).

Some results are proved in a stronger form. Lemma 9.3 holds for every path outside private nodes, and Lemma 9.4 for every path, live or not; Theorem 10.6 holds at every non-rendering role, and Theorems 10.5 and 10.6 hold for every closed term, not only for public ones; Proposition C.4 holds for every module, not only a rendering-only one. The proof of Lemma 10.2(i) at the witness and script roles uses the region argument, since Lemmas 9.3 and 9.4 are stated only at the protected roles. Theorem 9.7 locates the nodes of the normal form by their positions; the labels of Definition 7.1 are not typed.

Limitations. The elaboration of omitted grades is not implemented: the checker takes the core syntax, in which every binder carries its grade. What elaboration relies on is checked: an explicit grade is accepted exactly when $D(\sigma, \rho) \le q \le \theta(x)$, and $\theta(x)$ is accepted, so the maximal choice is well defined (Theorem 8.2). The inhabitation sentence of Theorem 10.5 rests on Proposition C.4, and the counterexamples of remarks 10.3 and 10.4 are checked, the second under the earlier gate rule, which the Lean library states as a separate signature judgment.

The calculus is intentionally intensional. It does not identify different histories with the same erased structure, and it does not add an extensional quotient of scripts. Those choices keep definitional equality decidable and make the provenance theorem literal rather than modulo an unspecified congruence; they are also the book's role discipline, under which equal derived values do not identify constructions, presentations, witnesses or transformation histories. Role-indexed universes, quotients and forced values, and certified inductive families remain open extensions.

15 Conclusion

The book sorts every datum into five roles and lets one rule carry most of the weight. The first part of this paper shows that the policy behind that rule is not a list of conventions. It is generated by four levels—the carrier, its carried structure, the instance, and the display—and four requirements the book states force it. It is path closed, so a construction can be checked one dependency at a time, and on every valid construction the firewall, recorded mediation, frame stability and presentation independence hold, for chains and for values.

The policy separates nine of the ten pairs of roles. The tenth pair, witnesses and scripts, shares the instance level because the book needs certificates about moves and moves that take certificates. Their rules separate them, and the separation is measurable: a history is a list of applications, a certificate only admits one, and when each role's data are counted one way, merging the two roles changes the book's counts. A state is a legal choice of values and so witness data, and every datum has one of the five roles.

The second part puts the roles into the judgement of a dependent type theory. Its min–max effect computes the least mediation on every path, a registered gate is the only source of a gated dependency, and its contraction leaves a record node with a module-owned event, so the static path and the dynamic provenance come from one rule. Renderings and fresh witness vocabularies are conservative, new evidence enables construction only on the record, and the book's frames, states, scripts, interpretation witnesses and renderings are its declarations. Both parts are checked in Lean, and checking the calculus corrected two of its rules.

A Complete declarative rule set

This appendix makes explicit the administrative rules used in the main text. There are no additional typing or reduction rules.

A.1 Raw and accepted judgments

First erase every policy-validity side condition and derive a *raw* judgment, written $\vdash^0_\rho$. Its effect is determined by the equations in sections 6.5 and 6.6. An accepted judgment is defined by

$$\frac{\mathfrak{S};\Gamma \vdash^0_\rho J \triangleright \varepsilon \qquad \mathsf{valid}_\rho(\varepsilon)}{\mathfrak{S};\Gamma \vdash_\rho J \triangleright \varepsilon}. \tag{Accept}$$

Every premise judgment in a rule is accepted, so illegal dependencies cannot be hidden in an intermediate type or term. In addition, every term judgment $\vdash_\rho t : A$ has an accepted formation derivation $\vdash_\rho A \;\; \mathsf{type}$; this is an implicit premise of each term rule below and ensures that grading the term never bypasses a dependency in its type.

The empty and extension context rules are

$$\frac{\mathfrak{S}\ \mathsf{wf}}{\mathfrak{S} \vdash \cdot\ \mathsf{ctx}} \tag{Ctx-Empty}$$

$$\frac{\mathfrak{S} \vdash \Gamma\ \mathsf{ctx} \quad \mathfrak{S};\Gamma \vdash_\rho A \;\; \mathsf{type} \triangleright \alpha \quad x \notin \Gamma}{\mathfrak{S} \vdash \Gamma, x :_\rho A\ \mathsf{ctx}}. \tag{Ctx-Extend}$$

A telescope judgment is defined sequentially:

$$\frac{\mathfrak{S} \vdash \Gamma\ \mathsf{ctx}}{\mathfrak{S};\Gamma \vdash \cdot\ \mathsf{tel} \triangleright \boldsymbol{\infty}} \tag{Tel-Empty}$$

and

$$\frac{\mathfrak{S};\Gamma \vdash \Delta\ \mathsf{tel} \triangleright \delta \quad \mathfrak{S};\Gamma,\Delta \vdash_\rho A \;\; \mathsf{type} \triangleright \alpha \quad x \notin \Gamma,\Delta}{\mathfrak{S};\Gamma \vdash \Delta, x :_\rho A\ \mathsf{tel} \triangleright \delta \oplus (\alpha \setminus \mathsf{dom}(\Delta))}. \tag{Tel-Extend}$$

Simultaneous term spines are checked from left to right:

$$\frac{\mathfrak{S};\Gamma \vdash_{\rho_i} t_i : A_i[\vec{t}_{<i}/\vec{x}_{<i}] \triangleright \varepsilon_i \quad (1 \le i \le n)}{\mathfrak{S};\Gamma \vdash \vec{t} : (x_1 :_{\rho_1} A_1, \ldots, x_n :_{\rho_n} A_n) \triangleright \bigoplus_i \varepsilon_i}. \tag{Spine}$$

A.2 Signature formation

The initial signature contains only the built-in formers Event, Origin, and Stamped:

$$\frac{}{\varnothing\ \mathsf{wf}}. \tag{Sig-Empty}$$

For each extension rule below, all introduced names are fresh.

$$\frac{\mathfrak{S}\ \mathsf{wf} \quad \mathfrak{S};\cdot \vdash \Delta\ \mathsf{tel} \quad D(\sigma,\rho) = 0 \text{ for each } (x :_\sigma C) \in \Delta \quad a \notin \mathfrak{S}}{\mathfrak{S}, a : (\Delta) \Rightarrow_\rho \mathsf{type}\ \mathsf{wf}}. \tag{Sig-Family}$$

The arguments of a family are ordinary dependencies of the type it forms; a family whose index needs a gate is not declared.

Use $\mathsf{gradefree}(A)$ as defined in section 6, and let $\mathsf{ordinaryHead}(A)$ mean that, after removing outer Π-binders, the result head is a declared family symbol. Primitive extension is

$$\frac{\mathfrak{S}\ \mathsf{wf} \quad \mathfrak{S};\cdot \vdash_\rho A \;\; \mathsf{type} \quad \mathsf{gradefree}(A) \quad \mathsf{ordinaryHead}(A)}{\mathfrak{S}, c :_\rho A\ \mathsf{wf}}. \tag{Sig-Primitive}$$

The head restriction reserves credentials for the next rule.

$$\frac{\mathfrak{S}\ \mathsf{wf} \quad \mathfrak{S};\cdot \vdash_\sigma A\ \ \mathsf{type} \quad \sigma \neq \mathsf{R} \quad \mathsf{gradefree}(A) \quad \mathsf{ordinaryHead}(A)}{\mathfrak{S}, a_d :_\sigma A, e_d :_{\mathsf{W}} \mathsf{Event}, o_d :_{\mathsf{W}} \mathsf{Origin}_A(e_d, a_d)\ \mathsf{wf}}. \tag{Sig-Certificate}$$

The conclusion is one atomic bundle: a later extension cannot introduce an origin constant for an event it does not own.

$$\frac{\mathfrak{S}\ \mathsf{wf} \quad \mathfrak{S};\cdot \vdash_{\mathsf{W}} t : A \quad \mathsf{public}(t) \quad \mathsf{gradefree}(A) \quad \mathsf{wdata}(A)}{\mathfrak{S}, e_q :_{\mathsf{W}} \mathsf{Event}, o_q :_{\mathsf{W}} \mathsf{Origin}_A(e_q, t)\ \mathsf{wf}}. \tag{Sig-Attest}$$

The empty context makes t closed. This rule introduces no payload constant and no reduction rule; it records authority for the exact public witness-data term already checked in $\mathfrak{S}$.

Remark A.1 (Why certificate heads remain restricted). If Sig-Certificate admitted a payload type $\mathsf{Stamped}_{\mathsf{W}}(W)$, its fresh payload constant would be a closed neutral normal term of stamped type. An old gate applied to it would be stuck, contradicting canonical credentials and record progress. Exact attestation does not create such a neutral payload, so it does not justify weakening the certificate rule.

For gate formation, suppose

$$\mathfrak{S};\cdot \vdash \Delta\ \mathsf{tel}, \quad \mathfrak{S};\Delta \vdash_\sigma A\ \ \mathsf{type}, \quad \mathfrak{S};\Delta \vdash_\rho B\ \ \mathsf{type},$$

where $\rho \in \{\mathsf{F}, \mathsf{P}\}$, $\sigma \neq \mathsf{R}$, each declaration role in Δ has demand 0 into ρ, $D(\sigma, \rho) \leq 1$, and the free variables of B are contained in Δ. The interface is grade free: $\mathsf{gradefree}(B)$, $\mathsf{gradefree}(A)$, and $\mathsf{gradefree}(C)$ for every type C of Δ (remark 10.4). Then

$$\frac{\text{the preceding premises} \qquad \text{grade-free interface} \qquad g \text{ fresh}}{\mathfrak{S}, g : \Delta; \mathsf{Stamped}_\sigma(A) \Rightarrow_\rho B[\mathsf{node}_g]\ \mathsf{wf}}. \tag{Sig-Gate}$$

This rule introduces the public atom g at role ρ, its public gate term former, and its private node former simultaneously. It introduces no general positive-grade constant.

A.3 Formation and typing summary

The complete type-former list is:

former	result role	effect
$a(\vec{t})$	ρ_a	$\mathbf{e}_a \oplus \bigoplus_i \varepsilon_i$
Event	W	∞
$\mathsf{Origin}_A(e, t)$	W	$\alpha \oplus \eta \oplus \theta$
$\mathsf{Stamped}_\sigma(A)$	W	α
$\Pi^q(x :_\sigma A).B$	ρ	$q \otimes \alpha \oplus (\beta \setminus x)$
$\Sigma(x :_\rho A).B$	ρ	$\alpha \oplus (\beta \setminus x)$

with the premises and restrictions of Ty-Fam, Ty-Event, Ty-Origin, Ty-Stamp, Ty-Pi, and Ty-Sigma. There is no universe, heterogeneous sum, quotient, identity type, or other implicit type former in $\mathsf{RATT}_{\mathsf{c}}$. The Ty-Pi row additionally requires the binder-edge check $D(\sigma, \rho) \leq q$.

The complete term-former list is:

term	principal effect
x	$\mathbf{e}_x$
c	$\mathbf{e}_c$
$\lambda^q x.t$	$\theta \setminus x \quad (q \le \theta(x))$
$f\,u$	$\phi \oplus (q \otimes \eta)$
(t,u)	$\theta \oplus \eta$
$\mathsf{fst}(p),\ \mathsf{snd}(p)$	π
$\mathsf{stamp}(t,e,o)$	$\theta \oplus \eta \oplus \omega$
$\mathsf{gate}_g(\vec{d};s)$	$\mathbf{e}_g \oplus \delta \oplus (1 \otimes \eta)$
$\mathsf{node}_g(\vec{d};t,e)$	$\mathbf{e}_g \oplus \delta \oplus (1 \otimes (\theta \oplus \eta))$

with exactly the typing premises in Var, Const, Lam, App, Pair, Fst, Snd, Stamp, Gate, and Node. Conv is the only rule that changes a displayed type. Node is available only in internal reducts.

B Reduction contexts and equality

Let $C[-]$ range over one-hole term contexts:

$$\begin{aligned} C ::= {} & [-] \mid \lambda^q(x :_\sigma A).C \mid C\,t \mid t\,C \mid (C,t) \mid (t,C) \mid \mathsf{fst}(C) \mid \mathsf{snd}(C) \\ & \mid \mathsf{stamp}(C,e,o) \mid \mathsf{stamp}(t,C,o) \mid \mathsf{stamp}(t,e,C) \\ & \mid \mathsf{gate}_g(\vec{d}_{<i}, C, \vec{d}_{>i}; s) \mid \mathsf{gate}_g(\vec{d}; C) \\ & \mid \mathsf{node}_g(\vec{d}_{<i}, C, \vec{d}_{>i}; t, e) \mid \mathsf{node}_g(\vec{d}; C, e) \mid \mathsf{node}_g(\vec{d}; t, C). \end{aligned}$$

If $t \longrightarrow t'$, then $C[t] \longrightarrow C[t']$ whenever both sides are well typed.

Term indices also reduce inside types. The type contexts containing a term hole are generated by

$$\begin{aligned} D ::= {} & a(\vec{t}_{<i}, C, \vec{t}_{>i}) \mid \mathsf{Origin}_A(C,t) \mid \mathsf{Origin}_A(e,C) \\ & \mid \mathsf{Origin}_D(e,t) \mid \mathsf{Stamped}_\sigma(D) \mid \Pi^q(x :_\sigma D).B \mid \Pi^q(x :_\sigma A).D \\ & \mid \Sigma(x :_\rho D).B \mid \Sigma(x :_\rho A).D. \end{aligned}$$

The notation Origin_D means that the payload type contains the hole. Reduction is closed under $D[-]$, and type contexts are also admitted in lambda annotations. This is every recursive position in the grammars of section 6; hence the compatible closure is fully determined.

Typed definitional equality is generated by

$$\frac{}{t \equiv t : A} \qquad \frac{t \equiv u : A}{u \equiv t : A} \qquad \frac{t \equiv u : A \quad u \equiv v : A}{t \equiv v : A} \qquad \frac{t \longrightarrow u}{t \equiv u : A},$$

together with congruence for each type and term context above and conversion between definitionally equal types. Equality has no additional extensional, script, quotient, proof-irrelevance, or event-erasing axiom.

C Reference decision procedure

Theorem 8.16 rests on the following procedure for a finite proposed signature and a proposed judgment. It works on the core syntax, whose binders carry their roles and grades; choosing an omitted grade $q = \theta(x)$ is elaboration, before it.

(1) Check the ordered signature entry by entry with Sig-Family, Sig-Primitive, Sig-Certificate, Sig-Attest and Sig-Gate, and a context entry by entry.

(2) Synthesize the role and effect of a type from its former.

(3) Synthesize a type for variables, constants, applications, projections, gates and nodes, and for abstractions and stamps whose body or payload synthesizes. Check pairs against a dependent sum, abstractions against a function type, stamps against a stamped type, and every other term by synthesis followed by a conversion test.

(4) Compute effects bottom-up by the rules, test $D(\sigma, \rho) \leq q \leq \theta(x)$ at every abstraction, $D(\sigma, \rho) \leq q$ at every function type, and that no origin has a rendering payload type; the validity of every conclusion follows.

(5) Normalize only when a conversion test requires it.

(6) For modules, compute the extension class and the qualified event namespace directly from the admitted declarations.

Definition C.1 (Bidirectional fragment)**.** A term *synthesizes* when it is a variable or a constant; an application whose function synthesizes and whose argument checks; a projection of a term that synthesizes; a gate or a node whose arguments and fields check; or an abstraction or a stamp whose body or payload synthesizes and whose other fields check. A term *checks* when it synthesizes, or is a pair of terms that check, an abstraction whose body checks, or a stamp whose fields check. A type, a context or a signature is in the fragment when the terms in its types check, and the term of each attestation checks.

Proposition C.2 (Declarative adequacy of the decision procedure)**.** *The procedure terminates on every input, and every judgment it accepts is derivable. It accepts every derivable signature, context, type formation and typing judgment of the bidirectional fragment, and every well-typed normal form, well-formed normal type and checked normal argument list is in the fragment.*

Proof. Each answer is built by the rules, so the procedure is sound, and it recurses on the size of the expression, so it terminates. Completeness is induction on the expression. Inversion of a derivation gives derivations of the parts; the induction hypothesis makes the recursive calls succeed; a synthesized type is convertible to every type of the term, so the conversion test succeeds by Theorem 8.16(i). In a normal term a synthesis position never holds an introduction form: an abstraction there would make a beta redex, a pair a projection redex, and otherwise the former would not match the type. □

Remark C.3 (Outside the fragment). A pair has no principal type: (t, u) has the type $\Sigma(x : A).B$ for every B with $u : B[t/x]$. A pair in a synthesis position is therefore rejected, although the judgment may be derivable. In the capture signature of section 12.6, $\mathsf{fst}\, s_0 : \mathsf{Table}(A)$ is derivable and the procedure rejects it. An earlier version of Proposition C.2 claimed completeness for every judgment, and of Theorem 8.16 decidable type checking for every term; both hold on the fragment.

Proposition C.4 (Signature strengthening)**.** *Let $\mathfrak{S} + \mathcal{M}$ be well formed, and let Γ be a context formed over $\mathfrak{S}$.*

(i) An old type in the fragment that is formed over $\mathfrak{S} + \mathcal{M}$ at role ρ is formed over $\mathfrak{S}$ at role ρ.

(ii) If an old term t that checks has $\mathfrak{S} + \mathcal{M}; \Gamma \vdash_\rho t : A \triangleright \varepsilon$, and A is formed over $\mathfrak{S}$ at role ρ, then $\mathfrak{S}; \Gamma \vdash_\rho t : A \triangleright \varepsilon$.

In particular, (ii) holds for every old normal term.

Proof. The procedure reads the signature only at the declarations that its subject names: it looks up a family, a constant or a gate at its position, and its conversion test normalizes, which does not read the signature. On an old subject every such position lies in $\mathfrak{S}$, where the two signatures have the same entries, so each answer of the procedure over $\mathfrak{S} + \mathcal{M}$ is an answer over $\mathfrak{S}$, with the same role, type and effect. By Proposition C.2 the procedure accepts the judgment over $\mathfrak{S} + \mathcal{M}$, and every answer it gives over $\mathfrak{S}$ is a derivation. Normal terms lie in the fragment by Proposition C.2. □

The procedure is implemented in Lean as a total, computable function on the core syntax whose answers carry their derivations, with the completeness proofs above (table 8). A serialized certificate format and a replay checker outside Lean remain open; such a certificate need only record the checked signature, principal effects, module class, event owners, and the normal forms used for conversion.

References

[1] S. Krivov, *Artform Calculus*, textbook, edition of 11 October 2026.

[2] S. Krivov, *Artform Calculus: proofs in Lean*, repository, 2026, https://github.com/skrivov/artform-calculus-textbook.

[3] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction – CADE 28*, LNCS 12699, Springer, 2021, 625–635.

[4] The mathlib Community, The Lean mathematical library, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2020, 367–381.

[5] S. Krivov, *Pattern Geometry in the Artform Calculus: Recognition, Generation, Measure, Scale, Dynamics, and Evaluation of Patterns in a Typed Calculus of Artforms*, working manuscript, 2026.

[6] P. Martin-Löf, *Intuitionistic Type Theory*, Bibliopolis, 1984.

[7] P. Dybjer, Internal type theory, in *Types for Proofs and Programs*, LNCS 1158, Springer, 1996, 120–134.

[8] D. Gratzer, G. A. Kavvos, A. Nuyts, and L. Birkedal, Multimodal dependent type theory, *Logical Methods in Computer Science* **17** (2021), no. 3, article 11.

[9] D. Gratzer, Normalization for multimodal type theory, *Logical Methods in Computer Science* **22** (2026), no. 1, article 27.

[10] A. Abel, N. A. Danielsson, and O. Eriksson, A graded modal dependent type theory with a universe and erasure, formalized, *Proceedings of the ACM on Programming Languages* **7** (2023), ICFP, article 220, 920–954.

[11] G. A. Kavvos, Modalities, cohesion, and information flow, *Proceedings of the ACM on Programming Languages* **3** (2019), POPL, article 20.

[12] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, A core calculus of dependency, in *Proceedings of the 26th ACM SIGPLAN–SIGACT Symposium on Principles of Programming Languages*, 1999, 147–160, https://doi.org/10.1145/292540.292555.

[13] A. Sabelfeld and D. Sands, Dimensions and principles of declassification, in *18th IEEE Computer Security Foundations Workshop*, 2005, 255–269, https://doi.org/10.1109/CSFW.2005.15.

[14] C. Theocharis and E. Brady, Type theory with erasure, in *11th International Conference on Formal Structures for Computation and Deduction (FSCD 2026)*, LIPIcs 378, article 31, 1–21, 2026, https://doi.org/10.4230/LIPIcs.FSCD.2026.31.

[15] H. Grodin, R. Li, and R. Harper, Abstraction functions as types: modular verification of cost and behavior in dependent type theory, *Proceedings of the ACM on Programming Languages* **10** (2026), POPL, article 31, 895–922.

[16] U. A. Acar, A. Ahmed, J. Cheney, and R. Perera, A core calculus for provenance, *Journal of Computer Security* **21** (2013), no. 6, 919–969.

[17] F. Blanqui, G. Dowek, E. Grienenberger, G. Hondet, and F. Thiré, A modular construction of type theories, *Logical Methods in Computer Science* **19** (2023), no. 1, article 12.